COURS MLOPS · FICHES DE RÉFÉRENCE

Labs Solutions Info

Utilité et rôle des outils, services et commandes — cours MLOps 36h

Document compagnon des labs

Onze fiches de référence — Lab 0 à Lab 10

Labs Solutions Info

Utilité et rôle des outils, services et commandes — cours MLOps 36h

Les documents « Commandes & outils » du cours listent les commandes sans expliquer à quoi elles servent. Ces fiches comblent ce manque : pour chaque lab, le rôle de chaque outil et service, les cibles Makefile et scripts, puis les commandes clés expliquées étape par étape, avec les pièges à connaître.

Sommaire

Lab	Titre

| Lab 0 | Setup de l'environnement | | Lab 1 | Versionner un projet ML end-to-end | | Lab 2 | Pipeline ML sur Kubeflow + Airflow | | Lab 3 | Gate de validation dans le pipeline | | Lab 4 | Packager le modèle de scoring en image Docker | | Lab 5 | Déployer le modèle de scoring sur KServe | | Lab 6 | CI/CD/CT de bout en bout sur K8s | | Lab 7 | Monitoring en production du modèle de scoring | | Lab 8 | Audit de gouvernance du modèle de scoring | | Lab 9 | RAG MLOps évalué et observé | | Lab 10 | Mini-projet MLOps |

Comment lire ces fiches

Chaque fiche suit le même plan : la stack et le rôle de chaque outil, les cibles Makefile et scripts du lab, les commandes clés expliquées (ce que fait la commande et pourquoi on l'utilise), puis les pièges rencontrés. Les commandes se lisent dans l'ordre de l'énoncé du lab correspondant.

Lab 0 — Setup de l'environnement · outils & commandes

Ce lab installe une fois pour toutes la stack commune à tous les modules : un cluster Kubernetes local (kind), un registry d'images local, un stockage objet S3 (SeaweedFS), un serveur MLflow partagé, un environnement Python reproductible (`uv`) et la chaîne d'outils MLOps (DVC, AWS CLI, Git). L'objectif est d'éviter la friction pendant les 36 h de cours. Le livrable est une capture de `make check-m0` à 100 % et la première pull request (branche `lab/m0-setup`) fusionnée.

Outils de la stack

Outil / service	Rôle / utilité
Docker	Moteur de conteneurs. Fait tourner les nœuds kind, le registry local, les outils en conteneur (client S3) et les services du cours. Version 27 minimum.
Docker Compose	Plugin `docker compose` : orchestration multi-conteneurs locale. L'ancien `docker-compose` v1 est abandonné.
kubectl	Client officiel pour piloter n'importe quel cluster Kubernetes (à ±1 version mineure du cluster, ici v1.37).
helm	Gestionnaire de paquets Kubernetes (charts : Prometheus, Argo CD aux labs suivants).
k9s	TUI pour explorer le cluster sans taper cinquante `kubectl get`. Raccourcis : `:pods`, `:svc`, `:ns`, `l` (logs), `s` (shell), `d` (describe), `Ctrl-D` (delete), `q` (quit).
kind (Kubernetes IN Docker)	Fait tourner un cluster K8s complet dans des conteneurs Docker : démarrage en ~1 min, suppression en une commande, isolation totale.
registry local (`localhost:5001`)	Registry d'images Docker local branché au cluster ; sert dès le Lab 2 (images du pipeline, puis de l'API). Port 5001 car 5000 est pris sur macOS (AirPlay) et réservé au port-forward MLflow.
SeaweedFS	Stockage objet S3-compatible (remplace MinIO, archivé en 2026). Backend DVC + artefacts MLflow + stockage générique. Même API S3 qu'AWS, donc portable.
MLflow	Serveur d'expériences partagé, dans le cluster. `backend store` = métadonnées (SQLite/PVC) ; `artifact store` = `s3://mlflow-artifacts`. Mode proxy : seul le serveur détient les identifiants S3.
uv	Gestionnaire Python rapide (Rust) : remplace pip + virtualenv + pip-tools + pyenv. Lockfile reproductible, gestion des versions Python intégrée.
Python 3.12	Interpréteur installé et piloté par `uv` ; version du venv du repo.
DVC	Versioning de données/modèles avec un remote S3. Adressage par contenu (MD5) : deux fichiers identiques ne sont stockés qu'une fois.

Outil / service	Rôle / utilité
AWS CLI (`aws-lab`)	Client S3 en conteneur, rien à installer (fonction définie dans `~/.bashrc`).
boto3	Client S3 Python, livré par le venv ; sert de secours si `aws-lab` échoue et est utilisé par `check.sh`.
Git	Versioning du code. Convention du cours : branche `lab/mX-<slug>` + pull request relue ; jamais de commit direct sur `main`.
GitHub	Héberge la copie (fork) du repo `mlops-cours-2026`, les branches et les pull requests ; authentification par clé SSH.
Images `kindest/node:v1.37.0`	Image des nœuds kind (control-plane + workers), ~1 Go.
Image `registry:3`	Registry Docker de référence, lancé comme conteneur `kind-registry`.
Image `ghcr.io/chrislusf/seaweedfs:4.48`	Image du service de stockage S3.
Image `ghcr.io/mlflow/mlflow:v3.16.1`	Image du serveur MLflow (boto3 installé au démarrage, ~1 Go).
Image `amazon/aws-cli:2.37.9`	Client AWS CLI utilisé en conteneur (Job `create-buckets`, fonction `aws-lab`).
Bibliothèques du socle	`mlflow`, `dvc[s3]`, `boto3`, `pandas`, `scikit-learn`, `feast` (Lab 1), `kfp` (Lab 2), `great-expectations`, `evidently` (Lab 3), `onnxruntime`, `fastapi`, `bentoml` (Lab 4), `locust`, `prometheus-client` (Lab 7), `fairlearn`, `shap`, `modelscan` (Lab 8).
Outils dev	Groupe `dev` installé par défaut : `ruff` (lint/format), `pytest`, `yamllint`.

Cibles Makefile et scripts

Cible / script	Rôle
`make help`	Liste les cibles disponibles.
`make cluster`	Crée le cluster kind (1 control-plane + 2 workers) et le registry local sur `localhost:5001`. Idempotent : le relancer ne casse rien.
`make storage`	Déploie SeaweedFS (namespace `storage`) + le Job créant les buckets `mlflow-artifacts` et `dvc-store`.
`make mlflow`	Déploie le serveur MLflow (namespace `mlflow`) + PVC `mlflow-data`.
`make pf`	Lance les port-forwards en arrière-plan : S3 → 8333, Filer → 8888, MLflow → 5000.
`make pf-stop`	Arrête les port-forwards.
`make prefetch`	Pré-télécharge toutes les images du cours (nœuds kind, MLflow, SeaweedFS, client AWS, registry).
`make check-m0`	Vérifie l'environnement (19 contrôles) et valide le Lab 0.
`make down`	Détruit le cluster et toutes les données.

Cible / script	Rôle
`make kfp` / `make kserve`	(Labs suivants) déploient Kubeflow Pipelines / KServe.
`infra/kind/create-cluster.sh`	Script idempotent appelé par `make cluster` : registry local + cluster + alias + réseau + ConfigMap.
`infra/versions.env`	Source unique des versions (kind, images, MLflow, KFP…) lue par les scripts.
`labs/m0-setup/check.sh`	Script de vérification exécuté par `make check-m0` (ne modifie rien).
`labs/m0-setup/smoke_mlflow.py`	Script du smoke test MLflow (logge paramètre, métriques, artefact).

Commandes clés, expliquées

Vérifier Docker (étape 1)

- `docker --version` / `docker compose version` — versions installées (Docker 27+, Compose v2/v5).
- `docker run --rm hello-world` — smoke test du moteur et du réseau Docker ; `--rm` supprime le conteneur après exécution.
- `docker info | grep -E 'CPUs|Total Memory'` — ressources allouées au moteur (viser 4 CPU / 6 Go mini).
- `sudo usermod -aG docker $USER` puis `newgrp docker` — ajoute l'utilisateur au groupe docker pour éviter `sudo`.

Installer les outils Kubernetes (étape 2)

- `curl -LO .../kubectl` + `chmod +x` + `sudo mv kubectl /usr/local/bin/` — installe le client kubectl (Linux x86_64) ; `brew install kubectl` sur macOS.
- `kubectl version --client` — vérifie la version du client (v1.36 minimum, à ±1 mineure du cluster).
- `curl -fsSL .../get-helm-4 | bash` / `brew install helm` — installe Helm 4 ; `helm version` contrôle.
- `wget .../k9s_linux_amd64.deb` + `sudo apt install ./k9s_linux_amd64.deb` / `curl -sS https://webinstall.dev/k9s | bash` / `brew install derailed/k9s/k9s` — installe k9s ; `k9s version`.
- `curl -Lo ./kind .../kind-linux-amd64` + `chmod +x` + `sudo mv ./kind /usr/local/bin/kind` / `brew install kind` — installe kind ; `kind version`.

Fork, clone et configurer Git (étape 3)

- `ssh-keygen -t ed25519 -C "votre.email@etu.fr"` — génère une clé SSH si nécessaire (à copier dans GitHub).
- `ssh -T git@github.com` — teste l'authentification SSH GitHub.
- `git clone git@github.com:VOTRE_USER/mlops-cours-2026.git` — clone votre copie (fork) en SSH.
- `git remote add upstream https://github.com/mlops-itbs-2026/mlops-cours-2026.git` — déclare le repo du cours pour récupérer les correctifs ; `git remote -v` vérifie.
- `git fetch upstream && git merge upstream/main && git push` — récupère une mise à jour du cours.

- `git config --global user.name "Prenom Nom"` / `git config --global user.email "votre.email@etu.fr"` — identité Git ; `--global` car le smoke test DVC crée un second dépôt jetable ; `git config --list | grep user` vérifie.
- `ls` / `make help` — tour du propriétaire : découvre `infra/`, `labs/`, `Makefile`, `dataset.yaml`, `docs/`, `pyproject.toml`, `uv.lock`, `.env.example`.
- `cp .env.example .env` / `cat .env` — crée le fichier d'environnement local (jamais commité).
- `set -a && source .env && set +a` — charge et exporte les variables de `.env` dans chaque nouveau terminal.

Créer le cluster kind + registry (étape 4)

- `make cluster` — crée le registry et le cluster (1 control-plane + 2 workers) ; premier lancement : télécharge l'image des nœuds (~1 Go).
- `kubectl cluster-info --context kind-mlops-lab` — adresse de l'API server.
- `kubectl get nodes -o wide` — contrôle les trois nœuds en état `Ready` (v1.37.0).
- `docker ps --filter name=kind-registry` — vérifie que le conteneur registry tourne.
- `curl -s http://localhost:5001/v2/_catalog` — le registry répond `{"repositories":[]}`.
- `kubectl config current-context` / `kubectl config use-context kind-mlops-lab` — vérifie / bascule le contexte courant.

Déployer SeaweedFS puis MLflow (étape 5)

- `make storage` + `kubectl -n storage get pods,pvc,svc` — déploie SeaweedFS ; attend un pod `Running`, PVC `Bound`, service `s3`.
- `kubectl -n storage logs job/create-buckets` — affiche `make_bucket: mlflow-artifacts`, `dvc-store` (le Job s'efface 10 min après sa fin, `ttlSecondsAfterFinished`).
- `make mlflow` + `kubectl -n mlflow get pods,pvc` — déploie MLflow ; attend un pod `Running` et PVC `Bound` (2 à 5 min au premier lancement).
- `make pf` — lance les port-forwards en arrière-plan (journaux dans `/tmp/mlops-port-forward/`).
- `kubectl --context kind-mlops-lab -n storage port-forward svc/s3 8333:8333 8888:8888` — port-forward manuel vers S3 et le filer.
- `kubectl --context kind-mlops-lab -n mlflow port-forward svc/mlflow 5000:5000` — port-forward manuel vers MLflow.
- `curl http://localhost:5000/health` — MLflow répond `OK`.
- `cat >> ~/.bashrc <<'EOF' ... aws-lab() { docker run --rm --network host ... } EOF` + `source ~/.bashrc` — définit le client S3 en conteneur.
- `aws-lab s3 mb s3://mlops-test` — crée un bucket.
- `aws-lab s3 ls` — liste les buckets (`dvc-store`, `mlflow-artifacts`, `mlops-test`).

Python, uv et pré-téléchargement (étape 6)

- `curl -LsSf https://astral.sh/uv/install.sh | sh` + `source ~/.bashrc` — installe uv ; `uv --version`.
- `uv python install 3.12` + `uv python list --only-installed` — installe et liste Python 3.12.
- `uv sync` — crée le venv et installe exactement les versions de `uv.lock`, désinstalle le reste. Ne pas faire `pip install`/`uv add` à la main.
- `source .venv/bin/activate` — active le venv ; ou préfixer par `uv run <cmd>` sans l'activer.

- `direnv` (`brew install direnv` / `sudo apt install direnv`, `eval "$(direnv hook bash)"`, `.envrc`, `direnv allow`) — activation automatique du venv et chargement de `.env`.
- `python --version` / `python -c "import sklearn, mlflow, dvc; print(...)"` / `dvc --version` — vérifications.
- `make prefetch` — pré-télécharge toutes les images du cours (10 à 20 min, sans surveillance).

Smoke tests MLflow et DVC (étape 7)

- `set -a && source .env && set +a` — charge `MLFLOW_TRACKING_URI` ; `echo $MLFLOW_TRACKING_URI` contrôle.
- `python labs/m0-setup/smoke_mlflow.py` — logge un run (paramètre, métriques, artefact `hello.txt`) via le mode proxy du serveur.
- `rm -rf /tmp/dvc-smoke && mkdir -p /tmp/dvc-smoke && cd /tmp/dvc-smoke` — crée un dossier jetable hors du repo.
- `git init && dvc init && git commit -m "dvc init"` — initialise un dépôt Git + DVC de test.
- `dvc remote add -d storage s3://mlops-test/dvcstore` — configure SeaweedFS comme remote DVC par défaut.
- `dvc remote modify storage endpointurl http://localhost:8333` — fixe l'endpoint (HTTP) dans `.dvc/config` (fichier commité).
- `dvc remote modify --local storage access_key_id mlops` / `... secret_access_key mlops-lab-secret` — identifiants stockés dans `.dvc/config.local` (ignoré par Git).
- `mkdir -p data && echo "hello mlops" > data/sample.txt` + `dvc add data/sample.txt` — crée et tracke un fichier avec DVC.
- `git add data/.gitignore data/sample.txt.dvc .dvc/config && git commit -m "Track sample.txt with DVC"` — commite les métadonnées DVC.
- `dvc push` — pousse l'objet vers S3 (`1 file pushed`).
- `aws-lab s3 ls --recursive s3://mlops-test/dvcstore/` — voit l'objet rangé sous `files/md5/...`.

Première pull request (étape 8)

- `git switch -c lab/m0-setup` — crée la branche du lab depuis `main` à jour.
- `git add NOTES.md labs/m0-setup/smoke_mlflow.py` / `git status` — prépare le commit ; `.env` ne doit jamais apparaître.
- `git commit -m "Lab 0: setup OK"` puis `git push -u origin lab/m0-setup` — pousse la branche et ouvre la PR.
- `gh pr create` — ouvre la PR en ligne de commande (facultatif, interactif).
- `git switch main && git pull` — revient sur `main` fusionnée ; `git log --oneline -3` confirme le merge.
- `rm NOTES.md labs/m0-setup/smoke_mlflow.py && git pull` — côté second membre du binôme (option A), récupère `main`.
- `make check-m0` — vérification automatique finale : attend `★ 19/19 vérifications OK — Lab 0 validé` (code retour = nombre d'échecs).

Pièges / points d'attention

- **Docker Desktop arrêté** : `connection refused` sur l'API server, `docker` introuvable dans WSL. Ce n'est pas un problème de cluster : démarrer Docker Desktop et attendre que les nœuds repassent `Ready`.
- **SeaweedFS en** `CrashLoopBackOff` : le Service s'appelle `s3`, Kubernetes injecte `S3_PORT=tcp://<clusterIP>:8333` et SeaweedFS ne sait pas le convertir en entier. Corrigé dans `infra/storage/storage.yaml` en figeant `S3_PORT: "8333"` (une variable du conteneur a priorité sur celle du Service).
- `make pf → make: *** [pf] Terminated` : avec `.ONESHELL`, un `pkill -f "port-forward svc/s3 8333"` matche le shell lui-même et le tue. Corrigé dans le Makefile en suivant les PIDs dans des fichiers.
- `aws-lab → exec format error` : anomalie du store d'images de Docker Desktop (mauvaise architecture). Contournement : créer/lister les buckets via `boto3` depuis la venv (`uv run --no-sync python - <<'PY' ... PY`).
- `kubectl -n storage logs job/create-buckets` → **« not found »** : normal, le Job (`ttlSecondsAfterFinished: 600`) est supprimé 10 min après sa fin. Se fier à `kubectl -n storage get pods` (pod `Completed`) et à la liste des buckets.
- **Dépôt DVC jetable :** `git commit` **refusé (identité)** : `/tmp/dvc-smoke` est un nouveau dépôt sans `user.name`/`user.email`. Renseigner l'identité locale (ou via la config globale de l'étape 3.4).
- **Port-forwards fragiles** : ils s'arrêtent si le pod redémarre, si le poste se met en veille ou si la session se ferme (symptôme : `Connection refused` sur `localhost:5000`/`:8333`). Remède : relancer `make pf`, sans risque, au début de chaque séance.
- **Ports déjà utilisés** : 80/443/30080 (Apache/nginx/IIS) ou 5001 (autre registry) font échouer `make cluster` ; arrêter le service ou commenter le mapping dans `infra/kind/kind-config.yaml`.
- **Ressources insuffisantes** : moins de 4 CPU ou 6 Go de RAM et le cluster rame ; augmenter dans Docker Desktop (Settings > Resources).
- **Windows/WSL2** : travailler dans le système de fichiers Linux (`/home/...`), jamais dans `/mnt/c/...`, sous peine de lenteur et de fichiers introuvables ; Docker Desktop doit utiliser le backend WSL2.
- `--network host` : sur Docker Desktop (macOS/Windows), l'activer ou remplacer par `http://host.docker.internal:8333` dans `aws-lab`.
- **Ne jamais commiter** `.env` : il contient des identifiants et est listé dans `.gitignore` ; le vérifier avec `git status`.
- **Ne pas recréer le cluster** : SeaweedFS et MLflow y vivent jusqu'au mini-projet ; `make down` (ou `kind delete cluster`) détruit toutes les données (datasets, expériences, modèles).
- `uv sync` **fait foi** : ne pas installer de paquets à la main (`pip install`, `uv add`) dans le repo du cours ; ils disparaîtraient au prochain `uv sync` ou modifieraient `uv.lock`.
- **ImagePullBackOff sur** `minio/minio` : images retirées de Docker Hub en 2026 (d'où le passage à SeaweedFS) ; utiliser `make storage` et le client `aws-lab`.

Fiche générée à partir de Lab0-Enonce.pdf et Lab0-Commandes.pdf — cours MLOps.

Lab 1 — Versionner un projet ML end-to-end · outils & commandes

Ce lab pose le projet fil rouge du cours : un modèle de *credit scoring* que l'on fera grandir de module en module. Il s'agit de versionner le projet de bout en bout — données avec DVC, 20+ runs tracés et comparés dans MLflow, promotion d'un modèle dans le Model Registry, features servies par Feast. Tout se fait à la racine du fork, sans nouveau dépôt.

Outils de la stack

Outil/service	Rôle / utilité
Git	Versionne le code, `dataset.yaml`, `params.yaml`, `dvc.yaml`, `dvc.lock` et les pointeurs DVC. C'est ce qui se partage entre postes.
DVC	Versionne les données et le modèle (contenu dans un remote S3, pointeurs dans Git). Décrit le pipeline `download → prepare → train → evaluate` et sait quand relancer une étape (`dvc repro`).
SeaweedFS (S3)	Stockage objet du cours (remplace MinIO). Héberge le bucket `dvc-store/scoring` pour les données versionnées et `mlflow-artifacts` pour les artefacts MLflow.
MLflow (tracking)	Serveur unique du cours : enregistre les runs (paramètres, métriques, tags, modèles) dans l'expérience `scoring`. Backend store SQLite sur volume persistant, artefacts en S3 via proxy.
MLflow Model Registry	Versionne les modèles promus (`scoring-model`) et gère les alias `@champion` / `@challenger` (remplacent les anciens « stages »).
Feast	Feature store local : une seule définition des features, deux accès (hors ligne pour l'entraînement, en ligne pour le serving) afin d'éviter le *training/serving skew*.
scikit-learn	Fournit le prétraitement (`ColumnTransformer`) et les estimateurs (régression logistique, forêts, gradient boosting, dummy).
Python 3.12 + uv	Environnement du cours ; le venv (`uv sync`) contient les dépendances et est réactivé à chaque séance.
kind / kubectl	Cluster Kubernetes local hébergeant SeaweedFS et MLflow (`kubectl get nodes` doit montrer 3 nœuds Ready).
Make	Raccourcis d'infrastructure et de vérification (`make up`, `make pf`, `make storage`, `make check-m1`…).
aws-lab	Client S3 du cours pour inspecter le contenu des buckets (`aws-lab s3 ls`).
GitHub CLI (`gh`)	Ouvre la pull request du lab (facultatif, alternative web).

Cibles Makefile et scripts

Cible / script	Rôle
`make up`	Démarre la stack du cluster (utilisé sur un poste neuf).
`make pf`	Lance les port-forwards : S3 `:8333` et MLflow `:5000`. À vérifier en début de séance.
`make storage`	Crée les buckets S3 du cours (idempotent) ; nécessaire avant un `dvc push`.
`make lint`	Vérifie le style du code (PEP 8). Utilisé dans les critères de propreté du code.
`make check-m0`	Vérifie le prérequis (Lab 0 : cluster, SeaweedFS, MLflow) ; doit être entièrement vert.
`make check-m1`	Vérifie automatiquement tout le Lab 1 (contrat, DVC, MLflow, registry, Feast) sans rien modifier.
`make check-mX`	Vérification générique de l'état des modules/labs.
`src/download.py`	Télécharge le dataset UCI (repli OpenML), décode les catégories et écrit le CSV décrit par `raw_path`.
`src/dataset_config.py`	Lit et valide `dataset.yaml`, expose une API typée (features, cible, split, préprocesseur, coût métier, seuil, groupes sensibles).
`src/prepare.py`	Crée l'identifiant de demande, fait le split stratifié et écrit `train.csv`, `test.csv` et la source hors ligne Feast `features.parquet`.
`src/train.py`	Entraîne un modèle (Pipeline complet) et trace le run MLflow (params, métriques, tags, modèle + signature).
`src/grid_search.py`	Lance 22 runs (un sous-processus par run) pour comparer les configurations dans MLflow.
`src/promote.py`	Enregistre un run dans le Model Registry et pose les alias `@challenger` / `@champion` selon le seuil de qualité.
`src/evaluate.py`	Relit le modèle local et écrit `reports/metrics.json`, versionné et comparable via DVC.
`features/definitions.py` et `features/feature_store.yaml`	Définitions Feast (entité, source, vue, service) construites depuis `dataset.yaml`.

Commandes clés, expliquées

Début de séance et branche de travail

- `cd ~/mlops-cours-2026` — se placer à la racine du fork : toutes les commandes s'y exécutent.
- `kubectl get nodes` — vérifie que le cluster kind est actif (3 nœuds Ready).
- `make pf` — ouvre les port-forwards S3 (`:8333`) et MLflow (`:5000`) vers le cluster.
- `source .venv/bin/activate` — active le venv du cours.
- `set -a && source .env && set +a` — charge les variables d'environnement (dont `MLFLOW_TRACKING_URI`) en les exportant.
- `git switch main && git pull` — part d'un `main` à jour.
- `git switch -c lab/m1-versioning` — crée la branche de travail du lab (convention du Lab 0).

Étape 1 — Contrat de données et téléchargement

- `python -m src.download` — télécharge et écrit le CSV brut ; lancé en module pour trouver le paquet `src` sans bricoler le `PYTHONPATH`.
- `head -3 data/raw/dataset.csv` — affiche l'en-tête et les premières lignes pour contrôler les colonnes.
- `wc -l data/raw/dataset.csv` — compte les lignes (1 000 demandes + 1 en-tête attendus).
- `python - <<'EOF' … EOF` — exploration rapide : dimensions, valeurs manquantes, déséquilibre de la cible et taux de *bad* par groupe sensible.
- `python -c "from src.dataset_config import load_config; print(load_config())"` — valide le contrat de données.
- `uv run pytest tests/test_dataset_config.py` — exécute les tests du contrat (utile si dataset personnalisé).

Étape 2 — Versionner les données avec DVC

- `dvc init` — initialise DVC dans le fork (crée `.dvc/`, `.dvc/config` et `.dvcignore`).
- `git status --short` — montre les nouveaux fichiers DVC à commiter.
- `dvc stage add -n download -d src/download.py -d src/dataset_config.py -p dataset.yaml:raw_path,target,features -o data/raw/dataset.csv python -m src.download` — déclare la stage `download` : `-d` = dépendances de code, `-p` = clés de paramètres surveillées, `-o` = sortie versionnée et ignorée par Git.
- `cat dvc.yaml` — relit la définition de la stage écrite par DVC.
- `dvc repro` — exécute la stage : DVC calcule les hashes et génère `dvc.lock`.
- `cat dvc.lock` — vérifie le hash des dépendances, les valeurs des paramètres et l'empreinte MD5 de la sortie.
- `dvc remote add -d storage s3://dvc-store/scoring` — enregistre et définit le remote par défaut du projet.
- `dvc remote modify storage endpointurl http://localhost:8333` — pointe le remote vers le SeaweedFS du cluster.
- `dvc remote modify --local storage access_key_id mlops` (et `secret_access_key mlops-lab-secret`) — stocke les identifiants hors Git, uniquement dans `.dvc/config.local`.
- `cat .dvc/config` / `cat .dvc/config.local` — vérifie que la config publique ne contient pas les secrets.
- `dvc push` — pousse les données vers S3 (rangées sous leur empreinte MD5).
- `aws-lab s3 ls --recursive s3://dvc-store/scoring/` — confirme la présence des objets dans le bucket.
- `git add .dvc/config .dvc/.gitignore .dvcignore dvc.yaml dvc.lock data/raw/.gitignore` — commite les pointeurs et la config DVC, pas les données.

Étape 3 — Vérifier le serveur MLflow

- `echo $MLFLOW_TRACKING_URI` — affiche l'URL du serveur MLflow utilisée par le client.
- `curl http://localhost:5000/health` — vérifie que le serveur répond (`OK`).
- `env -u AWS_ACCESS_KEY_ID -u AWS_SECRET_ACCESS_KEY python -c "import mlflow; print([e.name for e in mlflow.search_experiments()])"` — prouve que le client MLflow n'a besoin que de l'URL, pas des identifiants S3 (le serveur fait proxy).

Étape 4 — Préparation et entraînement instrumenté

- `python -m src.prepare` — crée l'identifiant de demande, le split stratifié et la source hors ligne Feast.
- `ls data/processed/` — vérifie la présence de `features.parquet`, `test.csv` et `train.csv`.
- `python -m src.train` — smoke test d'entraînement avec les paramètres par défaut de `params.yaml`.
- `python -m src.train --model random_forest --n_estimators 50 --max_depth 5` — smoke test avec surcharge d'hyper-paramètres (options de la CLI prioritaires sur `params.yaml`).
- `git add params.yaml src/prepare.py src/train.py` — commite les scripts et paramètres (jamais les données `data/processed/` ni `models/`).

Étape 5 — Grid search de 20+ runs

- `python -m src.grid_search` — lance les 22 runs (dummy, régression logistique, forêts, gradient boosting) dans l'expérience `scoring` ; chaque run est isolé et tagué `launched_by=grid_search`.
- `git add src/grid_search.py && git commit -m "feat: grid search (22 runs)"` — commite le script de grid search.

Étape 6 — Comparer et sélectionner dans MLflow

- UI `http://localhost:5000`, expérience `scoring` — explorer les runs (24 attendus), ajouter les colonnes `metrics.roc_auc`, `metrics.business_cost`, `tags.launched_by` et trier par `roc_auc`.
- `Compare` sur 3 runs (un de chaque type), vues `Parameters`, `Metrics`, `Parallel Coordinates Plot` — comprendre quels paramètres influencent le ROC AUC puis le coût métier.
- `python - <<'EOF' … client.search_runs([exp.experiment_id], order_by=["metrics.roc_auc DESC"], max_results=5) … EOF` — identifie les 5 meilleurs runs en CLI, triés par métrique primaire.

Étape 7 — Promouvoir dans le Model Registry

- `python -m src.promote` — enregistre le meilleur run comme version de `scoring-model`, pose `@challenger`, et `@champion` seulement si la métrique atteint `promotion_threshold` ; idempotent (pas de doublon).
- `python -m src.promote --filter "params.model = 'dummy'"` — illustre le seuil : le modèle naïf devient `@challenger` mais n'obtient pas `@champion` (roc_auc < seuil).
- `python - <<'EOF' … mlflow.sklearn.load_model("models:/scoring-model@champion") … EOF` — charge le champion par son alias et lit le seuil de décision rangé dans ses métadonnées : c'est ce que servira l'API.
- `git add src/promote.py && git commit -m "feat: promotion par alias avec seuil de qualité"` — commite le script de promotion.

Étape 8 — Pipeline DVC complet

- `dvc dag` — affiche le graphe `download → prepare → train → evaluate` déduit de `dvc.yaml`.
- `dvc repro` — relance uniquement les stages dont une dépendance (code ou paramètre) a changé.
- `dvc metrics show` — affiche les métriques de `reports/metrics.json`.
- `python -m src.promote --filter "tags.launched_by = 'dvc'"` — promeut le modèle produit par le pipeline (traçabilité complète : commit, données, paramètres, métriques).
- `sed -i 's/^ class_weight: balanced/ class_weight: none/' params.yaml` — modifie un paramètre pour tester la réactivité du pipeline (sur macOS : `sed -i ''`).

- `dvc status` — signale la stage impactée par la modification (`train: changed deps: params.yaml`).
- `dvc params diff` — montre l'écart de paramètre entre HEAD et l'espace de travail.
- `dvc metrics diff` — montre l'écart de métriques entre les versions (n'affiche que celles qui changent).
- `git checkout params.yaml` puis `dvc repro` — restaure la config commitée ; DVC reconnaît la combinaison déjà calculée et restaure les sorties depuis son *run-cache* (« skipping run »).
- `dvc push` — pousse les nouvelles sorties (données préparées, modèle) vers le remote.
- `git add dvc.yaml dvc.lock src/evaluate.py data/processed/.gitignore models/.gitignore reports/metrics.json` — commite le pipeline complet.

Étape 9 — Feature store local avec Feast

- `cd features; feast apply; cd ..` — importe `definitions.py` et enregistre entité, source, vue et service dans le registre ; crée la table SQLite en ligne.
- `feast materialize-incremental "$(date -u +%Y-%m-%dT%H:%M:%S)"` — copie les dernières valeurs de l'offline store vers l'online store (à lancer depuis `features/`).
- `python - <<'EOF' … store.get_historical_features(entity_df=…, features=store.get_feature_service("scoring_v1")) … EOF` — construit le jeu d'entraînement hors ligne et vérifie qu'il est identique à `train.csv`.
- `python - <<'EOF' … store.get_online_features(features=store.get_feature_service("scoring_v1"), entity_rows=[{id_col: first}]) … EOF` — sert les features en ligne par clé (quelques ms) pour une demande donnée.
- `grep -A8 'Lab 1 :' .gitignore` — vérifie que registre et online store (`features/data/`) ainsi que `mlruns/`/`mlflow.db` sont bien ignorés.
- `git add features/feature_store.yaml features/definitions.py` — commite les définitions Feast (pas les fichiers locaux générés).

Étape 10 — Reproductibilité et pull request

- `cat >> NOTES.md <<'EOF' … EOF` — documente la procédure de reproduction (livrable du lab).
- `git push -u origin lab/m1-versioning` — pousse la branche du lab.
- `gh pr create` — ouvre la pull request (alternative : « Compare & pull request » sur GitHub) ; relecture par le binôme puis fusion.
- `git switch main && git pull` — revient sur `main` à jour après fusion.
- `git clone … repro-test` puis `dvc pull; dvc repro; dvc metrics show` — simule un autre développeur sur le même poste : `dvc pull` restaure les données depuis le remote, `dvc repro` ne relance rien (tout « skipping »), mêmes métriques.
- `dvc repro --force` puis `dvc metrics diff --all` et `git diff --stat` — preuve forte : tout est recalculé (nouveau run MLflow), les métriques restent identiques ; seul `dvc.lock` change (hash de `models/model`).
- `dvc pull` (autre poste) puis `dvc repro; dvc metrics diff --all; dvc push` — sur un autre cluster le remote est vide : `dvc pull` échoue (missing cache files), on recalcule tout puis on remplit son propre remote.
- `make check-m1` — vérification automatique finale ; attendu `★ 18/18 vérifications OK — Lab 1 validé`.

Pièges / points d'attention

- `dvc push` : `NoSuchBucket` → le bucket `dvc-store` n'existe pas (job Lab 0 non terminé) ; lancer `make storage` (idempotent) puis `aws-lab s3 ls`.
- `dvc push` : `Could not connect to the endpoint URL` → port-forward S3 arrêté ; relancer `make pf`.
- `dvc push` : `Unable to locate credentials` → identifiants non configurés (clone frais, autre poste) ; `dvc remote modify --local storage access_key_id …` (et `secret_access_key`).
- `dvc push` : `SSL: WRONG_VERSION_NUMBER` → endpoint en `https://` alors que SeaweedFS écoute en HTTP ; corriger `endpointurl` en `http://localhost:8333`.
- `dvc repro` (download) : UCI indisponible puis erreur réseau → pas d'accès Internet ; récupérer `german.data` et lancer `python -m src.download --from-file german.data` puis `dvc commit download`.
- `ModuleNotFoundError: No module named 'src'` → script lancé par son chemin ou hors racine ; utiliser `python -m src.train` depuis `~/mlops-cours-2026`.
- `ModuleNotFoundError: mlflow / dvc / feast` (ou dans `dvc repro`) → venv non activé ; `source .venv/bin/activate` ou `uv run dvc repro`.
- `dvc repro` ne fait rien après modification → le fichier modifié n'est ni dans `deps` ni dans `params` ; vérifier `dvc.yaml`, `dvc status` dit ce que DVC surveille.
- `dvc add` : `cannot update 'dataset.csv': overlaps with an output of stage: 'download'` → un même fichier ne peut être à la fois `dvc add` et sortie de stage ; garder une seule méthode.
- MLflow : `Connection refused` sur `localhost:5000` → port-forward MLflow arrêté ; `make pf` puis `curl http://localhost:5000/health`.
- MLflow UI vide après un run → `MLFLOW_TRACKING_URI` non chargé (runs écrits dans `./mlruns` local) ; `set -a && source .env && set +a` puis supprimer `mlruns/`.
- `log_model` : « references untrusted types » → un type skops n'est pas dans `skops_trusted_types` ; ajouter le type cité (en vérifiant qu'il vient de scikit-learn ou numpy).
- `predict` : `Can not safely convert int64 to float64` → DataFrame lu par `pd.read_csv` passé tel quel ; utiliser `cfg.read_xy(chemin)` qui convertit les numériques.
- `promote` : « Aucun run avec la métrique … » → `metrics.primary` n'est pas une métrique loggée ou l'expérience est vide ; utiliser une métrique de `compute_metrics` et lancer `python -m src.grid_search`.
- `MlflowException: Registered model alias champion not found` → aucun modèle n'a encore passé le seuil ; lancer `python -m src.promote` et vérifier `metrics.promotion_threshold`.
- Ancien code `transition_model_version_stage` déprécié / `models:/…/Production` introuvable → les stages sont remplacés par les alias depuis MLflow 2.9 ; utiliser `set_registered_model_alias(...)` et `models:/scoring-model@champion`.
- `feast apply` : `ModuleNotFoundError: No module named 'src'` → `definitions.py` modifié sans la ligne `sys.path.insert(...)` ; garder `ROOT = …` et `sys.path.insert(0, str(ROOT))`.
- Feast : valeurs `None` en ligne → store en ligne non matérialisé ou fenêtre trop courte ; `feast materialize-incremental "$(date -u +%Y-%m-%dT%H:%M:%S)"` depuis `features/`.
- `dvc pull` : `KeyError: remote` / `no remote specified` → `.dvc/config` non commité ; `git add .dvc/config && git commit` dans le dépôt d'origine puis `git pull`.
- Autres points d'attention signalés : le même fichier ne peut être suivi à la fois par un `.dvc` et par une stage ; ne pas commiter `data/processed/` ni `models/` dans Git ; ne pas commiter le `dvc.lock` régénéré (seul le hash de `models/model` change, à cause de l'identifiant du run MLflow) ; un `pd.Timestamp.now()` dans `prepare.py` casserait la repro (horodatage fixe dans `params.yaml`) ;

messages MLflow `Detected uv project` / `Failed to resolve installed pip version` sans gravité (le venv uv n'a pas pip).

Fiche générée à partir de Lab1-Enonce.pdf et Lab1-Commandes.pdf — cours MLOps.

Lab 2 — Pipeline ML sur Kubeflow + Airflow · outils & commandes

Ce lab transforme le pipeline DVC du Lab 1 en un DAG Kubeflow Pipelines (KFP) production-ready (`scoring-pipeline` : ingest → validate → prepare → train → evaluate → register), puis en écrit une variante Airflow (`scoring_dag`) avec la même image et le même code. Il s'appuie sur le cluster kind du Lab 0, réutilise les scripts de `src/` via une image « projet » et trace chaque run dans MLflow. Objectif final : comparer les deux orchestrateurs (DX, scaling, debugging) et évaluer cette intégration de bout en bout.

Outils de la stack

Outil/service	Rôle / utilité
Kubeflow Pipelines (KFP 2.17)	Orchestrateur de DAG ML sur Kubernetes : API server, UI, SDK Python, upload/exécution du pipeline `scoring-pipeline`.
Argo Workflows (4.0)	Moteur d'exécution réel des DAG KFP (fournit la CRD `workflows.argoproj.io`).
Kubernetes / kind (`mlops-lab`)	Support d'exécution : 1 step KFP = 1 pod, 1 task Airflow = 1 pod. Cluster local en conteneurs Docker.
Docker + registry local `localhost:5001`	Build et stockage de l'image « projet » `scoring-pipeline:1.0.0`, tirée par le kubelet de chaque nœud.
SeaweedFS interne KFP (ns `kubeflow`)	Stockage S3 interne de KFP (bucket `mlpipeline`) : transporte et range les artefacts typés entre composants ; jamais manipulé à la main.
SeaweedFS du cours (ns `storage`)	S3 du cours (buckets `mlflow-artifacts`, `dvc-store`, `airflow-data`) utilisé par MLflow, DVC et le dossier de travail Airflow.
MLflow 3 (ns `mlflow`)	Tracking des runs et Model Registry : expérience `scoring`, versions de `scoring-model`, alias `@challenger` / `@champion`.
MySQL	Base des métadonnées de KFP.
ML Metadata (`metadata-grpc`)	Métadonnées et lineage des artefacts KFP.
Airflow 3.3 (Docker Compose)	Orchestrateur alternatif ; DAG équivalent lancé depuis `http://localhost:8081`.
KubernetesPodOperator (KPO)	Opérateur Airflow qui exécute chaque task dans un pod du cluster (même image que KFP).
DVC (Lab 1)	Référence métier : mêmes stages et mêmes commandes que les composants KFP.
`kfp` / `kfp-kubernetes` / `kfp.local`	SDK KFP, extensions K8s (`kubernetes.set_timeout`), exécution locale (`SubprocessRunner`).
`mlflow-skinny`, `scikit-learn`, `skops`, `pandas`, `pyarrow`, `boto3`	Dépendances figées dans l'image du pipeline (`pipelines/requirements.txt`).
`gh` (GitHub CLI)	Création facultative de la pull request de fin de lab.

Cibles Makefile et scripts

Cible / script	Rôle
`make cluster`	Prépare le cluster kind et le registry local `localhost:5001` (Lab 0).
`make pf`	Lance les port-forwards S3 (`:8333`) et MLflow (`:5000`) en arrière-plan.
`make mlflow`	Déploie le serveur MLflow du cours (ns `mlflow`).
`make kfp`	Applique les manifests KFP (script `infra/kfp/install.sh`, version lue dans `infra/versions.env`) puis attend la disponibilité des déploiements.
`make kfp-ui`	Port-forward de l'UI/API KFP sur `:8080` (journal `/tmp/mlops-port-forward/kfp-ui.log`).
`make pipeline-image`	Construit et pousse l'image `scoring-pipeline` dans le registry local.
`make airflow`	Installe Airflow en Docker Compose (`infra/airflow/install.sh`) et recopie le secret `s3-credentials`.
`make check-m1`	Vérifie que le Lab 1 est vert (prérequis).
`make check-m2`	Vérifie le Lab 2 (script `labs/m2-pipelines/check.sh`) sans rien modifier ; code retour = nombre d'échecs.
`infra/versions.env`	Source unique des versions (`KFP_VERSION`, `AIRFLOW_VERSION`).
`labs/m2-pipelines/starter/rapport-m2.md`	Modèle du rapport comparatif à recopier dans `reports/`.

Commandes clés, expliquées

Début de séance

- `cd ~/mlops-cours-2026` — se placer à la racine du fork, prérequis de toutes les commandes du lab.
- `kubectl get nodes` — confirmer que les 3 nœuds kind sont `Ready`.
- `make pf` — ouvrir les port-forwards S3 et MLflow (déjà actifs dans les terminaux suivants).
- `source .venv/bin/activate` puis `set -a && source .env && set +a` — activer le venv du cours et charger les variables d'environnement (dont les identifiants S3).

Branche de travail

- `git switch main && git pull` — partir de `main` à jour.
- `git switch -c lab/m2-pipelines` — créer la branche du lab, terminée par une pull request relue.

Étape 1 — Installer KFP

- `source infra/versions.env` — charger `KFP_VERSION=2.17.2`.

- `kubectl apply -k "$KFP_MANIFESTS/cluster-scoped-resources?ref=$KFP_VERSION"` — installer les ressources cluster-scoped (CRD, RBAC).
- `kubectl wait --for condition=established ... crd/workflows.argoproj.io` — attendre que les CRD (dont Argo) soient prêtes avant la suite.
- `kubectl apply -k "$KFP_MANIFESTS/env/platform-agnostic?ref=$KFP_VERSION"` — déployer la stack KFP dans le namespace `kubeflow`.
- `kubectl -n kubeflow get pods -w` — suivre le démarrage ; les 14 pods doivent finir `Running` (CrashLoopBackOff initial sur `ml-pipeline/metadata-grpc` normal).
- `make kfp-ui` — exposer l'UI puis ouvrir `http://localhost:8080`.
- `python -c "import kfp; print(kfp.__version__)"` — vérifier le SDK (attendu `2.17.0`).
- `curl -s http://localhost:8080/apis/v2beta1/healthz` — tester l'accès à l'API (`"multi_user":false`).

Étape 2 — Composants KFP, validation, image

- `python -m src.validate_data; echo "Code retour : $?"` — exécuter le gate de validation (code retour `0` si conforme).
- Génération d'un dataset corrompu (script inline) puis `python -m src.validate_data /tmp/dataset_bad.csv` — vérifier le fail-fast (colonne manquante → code retour `1`).
- `mkdir -p pipelines` / `touch pipelines/__init__.py` — créer le package des modules d'orchestration.
- Test de `run_stage` via script inline puis `cat /tmp/validation.json` — exécuter une stage dans un dossier temporaire sans modifier le repo.
- `docker build -f pipelines/Dockerfile --build-arg VERSION=1.0.0 -t localhost:5001/scoring-pipeline:1.0.0 .` — construire l'image « projet » (contexte = racine).
- `docker push localhost:5001/scoring-pipeline:1.0.0` — pousser l'image (sinon les pods restent en `ImagePullBackOff`).
- `curl -s http://localhost:5001/v2/scoring-pipeline/tags/list` — vérifier que le tag `1.0.0` est publié.
- `docker run --rm localhost:5001/scoring-pipeline:1.0.0 python -c "..."` — contrôler que l'image embarque bien `dataset.yaml` et `src/`.

Étape 3 — Écrire et compiler le pipeline

- `python -m pipelines.scoring_pipeline` — compiler le pipeline en `scoring_pipeline.yaml` (format IR).
- `ls -l scoring_pipeline.yaml` — constater le fichier compilé (~460 lignes).
- `grep -c "image: localhost:5001/scoring-pipeline:1.0.0" scoring_pipeline.yaml` — compter les exécuteurs sur l'image du projet (attendu `5` puis `6`).
- Script inline `local.init(runner=local.SubprocessRunner(use_venv=False))` — tester le DAG en local (sous-processus du venv, vrais artefacts) avant déploiement.

Étape 4 — Exécuter sur le cluster

- UI KFP : `Pipelines → Upload pipeline` (nom exact `scoring-pipeline`), puis `Create run` dans l'expérience `scoring`.
- `kubectl -n kubeflow get pods -w | grep scoring` — suivre les pods driver/impl de chaque step.

- Script inline `kfp.Client(...).create_run_from_pipeline_package(...)` — soumettre un run sans l'UI (même API que la CI du Lab 6).
- UI KFP onglet Input/Output — explorer les artefacts (`raw_data`, `checks`, `model`, `metrics`, matrice de confusion).

Étape 5 — Intégration MLflow

- `kubectl -n mlflow get pods,svc` — vérifier que MLflow tourne et expose le port `5000`.
- `kubectl -n kubeflow run mlflow-check ... python -c "..."` — tester la joignabilité de MLflow depuis un pod (via le DNS `mlflow.mlflow:5000`).
- `python -m pipelines.scoring_pipeline` — recompiler après avoir branché `register_model`.
- Script inline `MlflowClient()` — retrouver qui a produit `@challenger`/`@champion` et vérifier le tag `launched_by=kfp`.

Étape 6 — Retries, ressources, caching

- `python -m pipelines.scoring_pipeline` — recompiler avec `set_retry`, limites CPU/mémoire et timeouts.
- `grep -nE "maxRetryCount|resourceMemoryLimit|activeDeadlineSeconds" scoring_pipeline.yaml` — vérifier que les policies sont dans le YAML (`maxRetryCount: 3`, six `resourceMemoryLimit`, deux `activeDeadlineSeconds`).
- Relancer deux runs identiques dans l'UI — observer les steps data en `Cached` et le train ré-exécuté (caching désactivé).

Étape 7 — Airflow et DAG équivalent

- `source infra/versions.env` (dans `infra/airflow`) — charger `AIRFLOW_VERSION=3.3.2`, puis télécharger le `docker-compose.yaml` officiel.
- `mkdir -p ./logs ./plugins ./config ../../dags` — créer les dossiers attendus par Airflow.
- `echo "AIRFLOW_UID=$(id -u)" > .env` et `echo "AIRFLOW_IMAGE_NAME=apache/airflow:${AIRFLOW_VERSION}-python3.12" >> .env` — configurer Compose.
- `kind get kubeconfig --internal --name mlops-lab > kubeconfig` puis `chmod 600 kubeconfig` — donner aux conteneurs Airflow un accès interne au cluster.
- `kubectl create namespace airflow --dry-run=client -o yaml | kubectl apply -f -` — créer le namespace des pods de tasks (idempotent).
- `kubectl -n airflow create secret generic s3-credentials ... --dry-run=client -o yaml | kubectl apply -f -` — injecter les identifiants S3 du cours dans le namespace `airflow`.
- `docker compose up airflow-init` puis `docker compose up -d` et `docker compose ps` — initialiser puis démarrer Airflow (UI sur `:8081`).
- `kubectl -n airflow get pods -w` — suivre les pods des tasks au déclenchement du DAG.

Étape 8 — Tester et comparer

- `docker stats --no-stream` — mesurer la mémoire des nœuds kind vs conteneurs Airflow.
- `wc -l pipelines/components.py pipelines/scoring_pipeline.py dags/scoring_dag.py` — comparer la taille du code des deux orchestrateurs.
- `python -m pipelines.stage src.download --workspace /tmp/ws` puis `python -m pipelines.stage src.validate_data --workspace /tmp/ws` — rejouer une task Airflow en local (dossier au lieu de S3).

- `ls -R /tmp/ws` — vérifier le contenu produit (`data/raw/dataset.csv`, `reports/validation.json`).

Étape 9 — Rapport et commit

- `cp labs/m2-pipelines/starter/rapport-m2.md reports/rapport-m2.md` — copier le modèle de rapport à compléter.
- `git add src/validate_data.py pipelines/ dags/ reports/rapport-m2.md` — indexer les fichiers du lab.
- `git status --short` — vérifier qu'aucun fichier ignoré (`kubeconfig`, `scoring_pipeline.yaml`, `local_outputs/`) n'apparaît.
- `git commit -m "feat: pipeline KFP scoring-pipeline + DAG Airflow scoring_dag"` — commiter le travail.
- `git push -u origin lab/m2-pipelines` puis `gh pr create` — pousser et ouvrir la PR (relue par le binôme).
- `make check-m2` — lancer la vérification automatique finale (port-forwards actifs).

Pièges / points d'attention

- Le journal des problèmes fourni (`Lab 2 — Problèmes`) est un **modèle vide à remplir pendant le lab** (Symptôme / Cause probable / Solution) : aucun cas n'y est encore consigné.
- Ne pas confondre les **deux stockages S3** : le SeaweedFS du cours (`storage`, `s3.storage:8333`, secrets `s3-credentials`) et le SeaweedFS interne de KFP (`kubeflow`, `seaweedfs.kubeflow:8333`, bucket `mlpipeline`, secret `mlpipeline-minio-artifact`).
- Le **caching KFP** se base sur le nom de l'image, pas son contenu : reconstruire le même tag après un changement de code ressert l'ancien résultat (`imagePullPolicy: IfNotPresent`). Tout changement de code dans l'image = nouveau tag (`1.0.1`), reporté dans `PIPELINE_IMAGE`. Jamais `latest`.
- Une seule `limit` vaut aussi `request` : garder une `set_cpu_request("500m")` évite un pod `Pending (Insufficient cpu)` sur kind.
- Dans `components.py`, tout import doit être **dans** la fonction et aucune valeur par défaut ne doit dépendre d'une constante du module (sinon `NameError` dans le pod).
- `kfp.local` doit être lancé **depuis la racine du repo**, sinon `ModuleNotFoundError: No module named 'src'`.
- XCom ne supporte que les **petits objets** : y passer des clés/chemins S3, pas des DataFrames complets.
- `mlflow_uri` doit rester `http://mlflow.mlflow:5000` (autorisé par `--allowed-hosts`, Lab 0) ; les composants n'ont besoin d'aucun identifiant S3 (MLflow fait proxy).
- Le **gate de validation** doit rester fail-fast : si les données ne respectent pas `dataset.yaml`, le step échoue avant l'entraînement.
- Un step `OOMKilled` se corrige en augmentant son `set_memory_limit` (étape 6) ; les valeurs mesurées : train ~550 Mo, autres steps 200–500 Mo.
- Pod en `ImagePullBackOff` : image non poussée ou tag différent de `PIPELINE_IMAGE`.
- Ne jamais commiter ce que le lab régénère (`scoring_pipeline.yaml`, `kubeconfig`, `local_outputs/`, `reports/validation.json`).
- Ressources : compter ~6 Go de RAM pour KFP et ~4 Go pour Airflow ; arrêter Airflow (`docker compose down`) dès qu'il n'est plus utile.

- DAG Airflow `not found` : le fichier doit être dans `dags/` à la racine (attendre ~30 s le scan) ; en cas d'erreur d'import, utiliser `airflow dags list-import-errors`.
- Docker Compose trop ancien (`unknown tag !override`) : mettre à jour vers ≥ 2.24.4.

Fiche générée à partir de Lab2-Enonce.pdf et Lab2-Commandes.pdf — cours MLOps.

Lab 3 — Gate de validation dans le pipeline · outils & commandes

Ce lab ajoute au pipeline KFP `scoring-pipeline` du Lab 2 trois gates bloquants qui interdisent d'entraîner, d'évaluer ou d'enregistrer un modèle à partir de données ou de modèles non conformes au contrat. On y construit une suite Great Expectations, on simule et détecte un drift, puis on valide le modèle (Evidently + tests comportementaux) avant son enregistrement. Le principe reste le *fail-fast* : un gate en échec arrête le run et n'exécute aucun step dépendant.

Outils de la stack

Outil/service	Rôle / utilité
Great Expectations (GX) 1.23	Contrat de données : suite d'expectations générée depuis `dataset.yaml`, versionnée dans `gx/`, exécutée par un checkpoint. Produit Data Docs + rapport HTML.
Evidently 0.7	Rapports HTML de drift (`DataDriftPreset`) et de classification (`ClassificationPreset`), avec tests explicites (`gte`, `eq` + `Reference`).
scipy	Tests statistiques de drift : Kolmogorov-Smirnov (`ks_2samp`), chi2 (`chi2_contingency`), distance de Wasserstein.
pytest	Tests comportementaux du modèle (performance minimale, invariance, directionnels) ; rapport JUnit lisible par le gate.
KFP 2.17	Orchestrateur : exécute les composants dans l'image du projet, publie les artifacts (`HTML`, `Markdown`, `Metrics`) dans l'UI, gère les gates via `.after()`.
MLflow	Suivi des runs et Model Registry : `register-model` publie une nouvelle version de `scoring-model` seulement après validation.
DVC	Gestion des données et du modèle du Lab 1 (`dvc pull`, `dvc repro`).
Docker / registry local	Construit et pousse l'image du pipeline (`localhost:5001/scoring-pipeline:<tag>`), qui embarque `src/`, `gx/`, `tests/`.
Kubernetes / kubectl	Cluster `mlops-lab`, `kubectl get nodes`, port-forwards, accès au SeaweedFS interne.
SeaweedFS (S3)	Stockage des artifacts KFP (bucket `mlpipeline`), remplace MinIO depuis KFP 2.15 ; API S3 compatible.
MLflow RemoteStore / AWS CLI	Récupération des rapports HTML stockés dans SeaweedFS via un client S3 (`amazon/aws-cli`).
pandas / numpy / pyarrow	Manipulation des DataFrames, lecture/écriture CSV, `data/processed/features.parquet`.
Pandera (alternative)	Contrôle de schéma rapide en test unitaire (Lab 6), lui aussi généré depuis `dataset.yaml`.
Airflow	DAG `dags/scoring_dag.py` : mêmes composants, même image que KFP (`make check-m2` vérifie la cohérence).

Cibles Makefile et scripts

Cible / script	Rôle
`make pf`	Port-forwards S3 (:8333) et MLflow (:5000) en début de séance.
`make kfp-ui`	Expose l'UI et l'API KFP sur `http://localhost:8080`.
`make pipeline-image`	Reconstruit l'image du pipeline avec le nouveau tag.
`make check-m2`	Vérifie l'état du Lab 2 (pipeline existant) avant de démarrer.
`make check-m3`	Vérification automatique du Lab 3 (GX, drift, modèle, code, KFP, MLflow).
`src/validate_data.py`	Gate 1 : exécute la suite GX, écrit `reports/validation.json` + `reports/validation.html` + Data Docs ; code retour 1 si échec. `--build-suite` (re)génère la suite.
`src/simulate.py`	Fabrique des données de test : `corrupt` (bugs silencieux) et `drift` (nouvelle population selon `params.yaml`).
`src/detect_drift.py`	Gate 2 : compare référence/ courant par colonne (KS/χ^2 + PSI + Wasserstein), écrit `reports/drift.{json,md,html}` ; code retour 1 si drift.
`src/validate_model.py`	Gate 3 : Evidently (ClassificationPreset au seuil) + tests comportementaux pytest ; écrit `reports/model_validation.*` ; code retour 1 si un test échoue.
`pipelines/components.py`	Définit les composants KFP (`validate_data`, `detect_drift`, `validate_model`…) sur `PIPELINE_IMAGE`.
`pipelines/stage.py`	Lance une étape (`run_stage`) : copie entrées/sorties, propage l'échec (labo local recopie les rapports même en échec).
`pipelines/scoring_pipeline.py`	Pipeline principal avec les trois gates ; compile en `scoring_pipeline.yaml`.
`pipelines/test_bad_data.py`	Pipeline de test qui doit ÉCHOUER au gate 1 sur données corrompues.
`pipelines/drift_demo.py`	Pipeline de démonstration : drift simulé détecté par `detect_drift`.
`labs/m3-validation/check.sh`	Script appelé par `make check-m3` : contrôles locaux en dossiers temporaires.

Commandes clés, expliquées

Démarrage de séance

- `cd ~/mlops-cours-2026` — se placer à la racine du repo.
- `kubectl get nodes` — vérifie que les 3 nœuds du cluster sont `Ready`.
- `make pf` — ouvre les port-forwards S3 (:8333) et MLflow (:5000).
- `source .venv/bin/activate` — active le venv du cours.
- `set -a && source .env && set +a` — charge les variables d'environnement (export automatique).
- `dvc pull` — récupère données/modèle du Lab 1 (`dvc repro` si le remote DVC est vide).
- `make kfp-ui` — expose l'UI/API KFP (comme au Lab 2).

Branche de travail

- `git switch main && git pull` — repartir d'un `main` à jour.
- `git switch -c lab/m3-validation` — crée la branche dédiée du lab.

Étape 1 — Suite Great Expectations

- `python -c "import great_expectations as gx; print(gx.__version__)"` — vérifie GX dans l'environnement (attendu `1.23.2`, import lent ~3 s, normal).
- `python -m src.validate_data --build-suite` — (re)génère la suite depuis `dataset.yaml` et les données d'entraînement : écrit `gx/expectations/`, `gx/validation_definitions/`, `gx/checkpoints/`.
- `python -m src.validate_data; echo "Code retour : $?"` — valide les données brutes et affiche le résumé JSON ; `0` = conforme.
- `xdg-open gx/uncommitted/data_docs/local_site/index.html` — ouvre les Data Docs (site GX).
- `xdg-open reports/validation.html` — ouvre le rapport HTML autonome du gate.
- `python -m src.simulate corrupt data/raw/dataset.csv /tmp/dataset_corrupted.csv` — fabrique des données avec trois bugs silencieux (unité, code inconnu, cible manquante).
- `python -m src.validate_data /tmp/dataset_bad.csv; echo "Code retour : $?"` — validation de données corrompues : doit ressortir `success=false` et code 1.
- `git add gx/ src/validate_data.py src/simulate.py` — versionne la suite (contrat), le script et le simulateur (jamais `gx/uncommitted/`).
- `git commit -m "feat: contrat de données Great Expectations (gx/)"` — commit du contrat de données.

Étape 2 — Composant KFP et image 1.1.0

- `sed -i 's/scoring-pipeline:1.0.0/scoring-pipeline:1.1.0/' pipelines/components.py dags/scoring_dag.py` — passe le tag d'image (nouveau code = nouveau tag). Sur macOS : `sed -i '' 's/…/…/'`.
- `grep -n "scoring-pipeline:" pipelines/components.py dags/scoring_dag.py` — vérifie que les deux fichiers portent bien le nouveau tag.
- `docker build -f pipelines/Dockerfile --build-arg VERSION=1.1.0 -t localhost:5001/scoring-pipeline:1.1.0 .` — reconstruit l'image (embarque `gx/`, `tests/`, nouvelles dépendances).
- `docker push localhost:5001/scoring-pipeline:1.1.0` — pousse l'image dans le registry local.
- `docker run --rm localhost:5001/scoring-pipeline:1.1.0 python -c "import great_expectations as gx, evidently; print(gx.__version__, evidently.__version__)"` — vérifie que GX/Evidently sont bien dans l'image.
- `docker run --rm localhost:5001/scoring-pipeline:1.1.0 ls gx` — vérifie que la suite versionnée (sans `uncommitted/`) est dans l'image.

Étape 3 — Compiler et tester en local

- `python -m pipelines.scoring_pipeline` — compile le pipeline en `scoring_pipeline.yaml`.
- `wc -l scoring_pipeline.yaml` — compte les lignes du YAML compilé.
- `grep -c "image: localhost:5001/scoring-pipeline:1.1.0" scoring_pipeline.yaml` — vérifie que chaque exécuteur utilise la bonne image (6 attendus à ce stade).

- Bloc `kfp.local` (`local.init(runner=local.SubprocessRunner(use_venv=False))` puis `scoring_pipeline(mlflow_uri="http://localhost:5000")`) — exécute le pipeline en local sans cluster.

Étape 4 — Tester le gate sur données corrompues

- `python -m pipelines.test_bad_data` — compile le pipeline de test (`test_bad_data.yaml`).
- Bloc `kfp.local` avec `raise_on_error=False` — lance `bad_data_pipeline()` : l'échec du gate est affiché sans interrompre le script.
- `ls local_outputs/scoring-bad-data-test-*/validate-data/` — vérifie que `html` et `report` sont bien produits en local même en échec.

Étape 5 — Détecter le drift

- `cat >> params.yaml <<'EOF' … EOF` — ajoute la section `drift` (seuils `ks_alpha`, `psi_threshold`, `drift_share` et scénario `simulation`).
- `python -m src.detect_drift; echo "Code retour : $?"` — compare entraînement et test (défauts) : tableau par colonne, statut du gate.
- `python -m src.simulate drift data/processed/test.csv /tmp/test_drifted.csv` — applique le scénario de drift au jeu de test.
- `python -m src.validate_data /tmp/test_drifted.csv | grep -E '"(success|n_failed)"'` — montre que les données déplacées respectent encore le contrat (le drift n'est pas une violation de contrat).
- `python -m src.detect_drift data/processed/train.csv /tmp/test_drifted.csv; echo "Code retour : $?"` — détecte le drift simulé (code retour 1).
- `python -m pipelines.drift_demo` — compile le pipeline de démonstration de drift (`drift_demo.yaml`).
- Bloc `kfp.local` sur `drift_pipeline()` — exécute la démo : drift signalé sans bloquer.

Étape 6 — Valider le modèle

- `python -m pytest -q tests/test_model_behavior.py` — lance les tests comportementaux (8 attendus).
- `python -m src.validate_model; echo "Code retour : $?"` — Evidently + pytest : rapport `reports/model_validation.*`, code `0` si le modèle est validé.
- `python -m pipelines.scoring_pipeline` — recompile le pipeline final avec les trois gates.
- `wc -l scoring_pipeline.yaml` — compte les lignes du pipeline final.
- `sed -i 's/scoring-pipeline:1.1.0/scoring-pipeline:1.2.0/' pipelines/components.py dags/scoring_dag.py` — passe au tag 1.2.0.
- `docker build -f pipelines/Dockerfile --build-arg VERSION=1.2.0 -t localhost:5001/scoring-pipeline:1.2.0 .` puis `docker push` — reconstruit (rapide, dépendances en cache) et pousse l'image 1.2.0.
- `python -m pipelines.drift_demo` puis `grep -c "image: …:1.2.0" scoring_pipeline.yaml` — recompile la démo et vérifie le tag.

Étape 7 — Exporter et récupérer les rapports

- `kubectl -n kubeflow port-forward svc/seaweedfs 9090:8333 >/tmp/kfp-s3.log 2>&1 &` — port-forward vers le S3 interne de KFP.

- `KFP_S3_KEY=$(kubectl -n kubeflow get secret mlpipeline-minio-artifact -o jsonpath='{.data.accesskey}' | base64 -d)` — lit la clé S3 de KFP depuis le secret.
- `KFP_S3_SECRET=$(kubectl -n kubeflow get secret mlpipeline-minio-artifact -o jsonpath='{.data.secretkey}' | base64 -d)` — lit la clé secrète S3.
- Fonction `kfp-s3()` — enveloppe le client `amazon/aws-cli` (réseau host, montage `reports/kfp:/data`) pointé sur `--endpoint-url http://localhost:9090`.
- `kfp-s3 s3 ls s3://mlpipeline/v2/artifacts/scoring-pipeline/` — liste les runs stockés.
- `kfp-s3 s3 ls --recursive "s3://…/$RUN_ID/" | grep /html` — liste les rapports HTML d'un run.
- `kfp-s3 s3 cp "s3://mlpipeline/$KEY" /data/drift.html` — télécharge un rapport HTML précis.
- `kfp-s3 s3 cp --recursive "s3://…/$RUN_ID/" "/data/$RUN_ID/"` — récupère tout le run d'un coup.
- `pkill -f "port-forward svc/seaweedfs"` — arrête le port-forward.

Commit et push final

- `git add params.yaml gx/ src/ tests/ pipelines/ dags/scoring_dag.py` — stage les sources et le contrat.
- `git status --short` — vérifie qu'aucun rapport, YAML compilé ou `gx/uncommitted/` n'est inclus.
- `git commit -m "feat: gates de validation (GX, drift, modèle) dans scoring-pipeline"` — commit des trois gates.
- `git push -u origin lab/m3-validation` — pousse la branche.
- `gh pr create` — ouvre la PR (ou « Compare & pull request » sur GitHub) ; le partenaire relit, notamment la suite GX.
- `make check-m3` — vérification automatique complète du lab.

Pièges / points d'attention

- L'import de `great_expectations` prend ~3 s : ce n'est pas un bug.
- Le code GX 0.x ne fonctionne pas en 1.x : utiliser `context.data_sources`, `context.suites`, `ValidationDefinition`, `Checkpoint.run()`.
- Construire une suite exige un contexte actif : appeler `gx.get_context()` d'abord.
- `Suite absente` : lancer `--build-suite`, `git add gx/`, et copier `gx/` dans l'image avec un nouveau tag.
- `FileNotFoundError: data/processed/train.csv` : données du Lab 1 absentes (`dvc pull` ou `dvc repro`).
- La suite peut valider des données corrompues si moins de 1 % de lignes sont touchées (`mostly=0.99`) ou si elle a été regénérée depuis ces données : corrompre plus de lignes, ne jamais régénérer la suite depuis les données à valider.
- Un `expect_column_values_to_be_in_set` en échec = modalité inconnue : corriger la source ou faire évoluer le contrat (revue du `git diff gx/`).
- `validate-data` en `OOMKilled` : la limite de 512 Mi du Lab 2 est trop basse pour GX → `set_memory_limit("1Gi")`.
- `ModuleNotFoundError: great_expectations` dans un pod : l'image utilisée est encore 1.0.0 → mettre à jour le tag, repousser l'image, recompiler et uploader.

- `pytest: No module named 'src'` : lancer depuis la racine du repo (`python -m pytest` ou `uv run pytest`).
- Tests comportementaux tous *skipped* : pas de modèle local (`dvc pull` / `dvc repro`).
- Evidently `AttributeError: 'list' object has no attribute 'items'` : bogue de `ClassificationPreset(include_tests=False)` en 0.7.23 → désactiver les tests métrique par métrique (listes vides).
- Tests par défaut d'Evidently (« mieux qu'un modèle naïf ») en FAIL (TNR/FPR) : inadaptés au seuil coût-optimal → tests explicites seulement.
- Erreurs Deepchecks (`np.Inf`, `max_error`) : Deepchecks 0.19 incompatible NumPy 2 / scikit-learn ≥ 1.8 → remplacé par Evidently + pytest.
- `prep_task` démarre avant `validate` : `.after()` oublié.
- Un `sys.exit(1)` n'arrête pas le step si l'exception est attrapée : laisser remonter l'erreur pour que `run_stage` fasse échouer le step.
- Un artifact HTML ne s'affiche pas : n'utiliser qu'un seul fichier commençant par `<!DOCTYPE html>`, en UTF-8 (le rapport Evidently ≈ 4 Mo met quelques secondes).
- Pas de rapport HTML sur un step `validate-data` en échec : KFP ne publie que les sorties des steps réussis → lire les logs ou rejouer en local (`kfp.local`).
- KS p-value toujours très basse sur gros dataset : normal, d'où l'exigence d'un PSI ≥ 0,25.
- Valeurs PSI extrêmes : cases vides/trop fines → déciles de référence, ε = 1e-4, une case par modalité pour les discrètes.
- Evidently signale un drift non confirmé par le tableau : son verdict ne tient qu'à la p-value → comparer la colonne PSI (`DataDriftPreset(method="psi")` pour un verdict par magnitude).
- `detect-drift` échoue dans `scoring-pipeline` : le test ne ressemble pas à l'entraînement (petit dataset, split non stratifié) → lire le tableau, revoir le split, ajuster les seuils en le justifiant.
- `aws : Could not connect to the endpoint URL` : port-forward SeaweedFS absent.
- `aws : InvalidAccessKeyId` : identifiants du S3 du cours (mlops) au lieu de ceux de KFP → lire `mlpipeline-minio-artifact`.
- `aws s3 ls` sans résultat : mauvais préfixe (`v2/artifacts/<pipeline>/<run_id>/`), vérifier l'id dans l'URL de l'UI.
- Le split aléatoire et stratifié du Lab 1 ne crée aucun drift : Evidently peut signaler à tort des modalités rares à cause de son χ^2 sur effectifs → toujours accompagner un test d'une mesure de magnitude.
- Un gate ne se contente pas d'un avertissement : si l'exception est attrapée sans `raise`, le step reste vert et le run continue.

Fiche générée à partir de Lab3-Enonce.pdf et Lab3-Commandes.pdf — cours MLOps.

Lab 4 — Packager le modèle de scoring en image Docker · outils & commandes

Ce lab transforme le modèle MLflow du Lab 1 (`scoring-model@champion`) en une image Docker portable, optimisée (< 500 Mo), scannée et prête pour la production. On y exporte le modèle en ONNX, on l'expose via une API FastAPI générique, on compare les latences scikit-learn vs ONNX, on produit une variante BentoML, on scanne images et modèles, puis on pousse l'image sur le registry local du cours.

Outils de la stack

Outil/service	Rôle / utilité
Docker Engine	Construit et exécute les images et conteneurs de serving.
Dockerfile multi-stage (BuildKit)	Sépare un stage « builder » (installation des dépendances) d'un stage « runtime » minimal, pour réduire la taille de l'image finale.
`.dockerignore` (`<Dockerfile>.dockerignore`)	Exclut fichiers et secrets du contexte de build ; la liste blanche du Dockerfile optimisé ne ré-inclut que le strict nécessaire.
ONNX / ONNX Runtime	Format de graphe statique (protobuf) et moteur d'inférence ; exécute le modèle sans scikit-learn ni MLflow.
skl2onnx	Convertit le pipeline scikit-learn (prétraitement inclus) en graphe ONNX.
MLflow Registry	Source du modèle (`models:/scoring-model@champion`) et de ses métadonnées (version, run_id, seuil).
SeaweedFS (S3)	Stockage d'artefacts du cluster, proxifié par le serveur MLflow : aucun identifiant S3 côté client.
FastAPI	Framework HTTP de l'API de scoring ; génère validation et doc OpenAPI (`/docs`).
uvicorn	Serveur ASGI qui sert l'application FastAPI.
Pydantic (`create_model`)	Construit dynamiquement le schéma d'entrée à partir de `dataset.yaml` (validation stricte).
BentoML	Alternative de serving : enregistre le modèle dans son model store et le sert/empaquette.
Locust	Test de charge de l'API (rejoue des lignes réelles de test).
Trivy	Scanne les images Docker à la recherche de vulnérabilités OS et libs (sévérités HIGH/CRITICAL).
ModelScan	Scanne les fichiers modèles à la recherche d'opérateurs dangereux (ex. `os.system` dans un pickle).

Outil/service	Rôle / utilité
skops	Format de sérialisation de MLflow 3 ; reconstruit des objets Python uniquement dans une liste de types autorisés.
Hadolint	Linter de Dockerfile (versions épinglées, cache pip, etc.).
Registry local (`kind-registry`, `registry:3`)	Registry du cours sur `localhost:5001`, branché au cluster : les nœuds kind y tirent les images.
kubectl / cluster kind	Vérifie que le cluster voit et démarre l'image poussée (smoke test).
uv	Gestionnaire d'environnement : `uv sync`, `uv run` (socle incluant skl2onnx, onnxruntime, fastapi, bentoml, modelscan, locust).

Cibles Makefile et scripts

Cible / script	Rôle
`make pf`	Lance les port-forwards du cluster : rend le serveur MLflow accessible sur `http://localhost:5000`.
`make cluster`	Démarre le cluster kind et le registry local `kind-registry` (`registry:3`) sur `localhost:5001`.
`make check-m4`	Vérification automatique du lab : modèle exporté, registry, image (< 500 Mo, non-root, HEALTHCHECK), conteneur, livrables.
`packaging/app/export_onnx.py`	Récupère `scoring-model@champion` et l'exporte en `model.onnx` + `metadata.json`.
`packaging/app/verify_onnx.py`	Compare les probabilités ONNX vs scikit-learn sur `data/processed/test.csv` (tolérance 1e-4).
`packaging/app/serve.py`	API FastAPI de scoring (seul script embarqué dans l'image).
`packaging/sample_request.py`	Fabrique un corps JSON d'exemple à partir de vraies lignes de test.
`packaging/app/benchmark.py`	Mesure P50/P95/P99 : scikit-learn vs ONNX (1 ligne / lot de 100) ; écrit `reports/benchmark.md`.
`packaging/locustfile.py`	Scénario de test de charge rejouant des lignes aléatoires de test.
`packaging/bento/save_model.py`	Copie le modèle MLflow exporté dans le model store BentoML.
`packaging/bento/service.py`	Variante BentoML de l'API (même route `/predict`, port 3000).
`packaging/bento/bentofile.yaml`	Description du Bento (service, dépendances, version Python, image Docker).
`packaging/scan-demo/make_pickles.py`	Fabrique deux pickles de démo (dont un piégé) pour illustrer ModelScan.
`packaging/Dockerfile.naive`	Dockerfile baseline volontairement naïf (grand et non sécurisé).
`packaging/Dockerfile`	Dockerfile multi-stage optimisé.

Cible / script	Rôle
`packaging/requirements.txt`	Dépendances figées de l'image de serving uniquement.
`labs/m4-packaging/check.sh`	Script lancé par `make check-m4` (ne modifie rien, supprime son conteneur de test).

Commandes clés, expliquées

Préparation et branche

- `cd ~/mlops-cours-2026` puis `set -a && source .env && set +a` : se placer à la racine du fork et exporter les variables d'environnement (dont `MLFLOW_TRACKING_URI`).
- `make pf` : établit les port-forwards, indispensable pour que MLflow réponde sur `localhost:5000`.
- `uv sync` : installe/synchronise l'environnement avec le socle du cours.
- `mkdir -p packaging/app packaging/model packaging/bento packaging/scan-demo reports` : crée l'arborescence de travail du lab.
- `git switch main && git pull` puis `git switch -c lab/m4-packaging` : crée la branche de travail dédiée depuis `main` à jour.

1.1 Vérifier le modèle du Lab 1

- `uv run python -c "from mlflow import MlflowClient; mv = MlflowClient().get_model_version_by_alias('scoring-model', 'champion'); print(mv.name, mv.version, mv.run_id)"` : confirme l'existence du champion et affiche sa version (≥ 7) et son `run_id`.

1.2 Exporter le modèle en ONNX

- `uv run python packaging/app/export_onnx.py` : charge `models:/scoring-model@champion`, le convertit avec skl2onnx (entrées dérivées de `dataset.yaml`) et écrit `packaging/model/model.onnx` + `metadata.json` (version, run_id, seuil, empreinte SHA-256).

1.3 Vérifier que ONNX = sklearn

- `uv run python packaging/app/verify_onnx.py` : compare les probabilités ONNX et scikit-learn sur tout le test set ; doit afficher `OK: predictions match.` sans `AssertionError`.

2.2 Test local de l'API

- `uv run python packaging/sample_request.py > /tmp/request.json` : génère une requête JSON valide depuis une vraie ligne de test ; `cat /tmp/request.json` l'affiche.
- `uv run python -m uvicorn app.serve:app --app-dir packaging --port 8000 &` : lance l'API localement ; `--app-dir packaging` ajoute `packaging/` au chemin d'import et `python -m uvicorn` garde la racine du repo importable.
- `sleep 3` puis `curl -s http://localhost:8000/health` : attend le démarrage et vérifie la sonde de vie (`{"status":"ok","model_loaded":true}`).
- `curl -s -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d @/tmp/request.json` : envoie une demande et récupère probabilité + décision au seuil.
- `curl -s -X POST http://localhost:8000/predict ... -d '{"instances": [{"duration": "douze"}]}' | python3 -m json.tool | head -12` : provoque une erreur 422 détaillée (mauvais type, colonnes manquantes).

- `uv run python packaging/sample_request.py -n 3 | curl -s -X POST ... -d @-` : teste plusieurs demandes dans une seule requête (`instances` avec `-n 3`).
- `kill %1` : arrête le serveur local en arrière-plan.

3. Dockerfile naïf (baseline)

- `docker build -f packaging/Dockerfile.naive -t scoring-api:naive .` : build baseline depuis la racine du repo (contexte `.`), Dockerfile désigné par `-f`.
- `docker images scoring-api:naive --format "{{.Size}}"` : relève la taille (1,3 à 1,6 Go) pour le rapport.
- `docker run --rm scoring-api:naive id` : montre que le serveur tourne en `uid=0(root)`.
- `docker run --rm scoring-api:naive cat .env` : prouve que `COPY . /app` embarque les secrets du `.env`.

4. Dockerfile multi-stage optimisé

- `MODEL_VERSION=$(uv run python -c "import json; print(json.load(open('packaging/model/metadata.json'))['model_version'])")` : lit la version du modèle pour la passer en LABEL/`build-arg`.
- `docker build -f packaging/Dockerfile -t scoring-api:optimized --build-arg VERSION=1.0.0 --build-arg MODEL_VERSION="$MODEL_VERSION" .` : build l'image optimisée multi-stage avec métadonnées.
- `docker images scoring-api:optimized --format "{{.Size}}"` : vérifie la taille cible (< 500 Mo).
- `docker run -d --rm -p 8000:8000 --name scoring scoring-api:optimized` puis `sleep 5 && curl -s http://localhost:8000/health` : démarre le conteneur et teste la sonde.
- `curl -s -X POST http://localhost:8000/predict ... -d @/tmp/request.json` : vérifie que l'image renvoie la même prédiction qu'en local.
- `docker run --rm scoring-api:optimized id` : confirme l'utilisateur non-root (`uid=10001(ml)`).
- `docker inspect --format '{{.State.Health.Status}}' scoring` : lit l'état du HEALTHCHECK (`healthy`).
- `docker inspect --format '{{json .Config.Labels}}' scoring-api:optimized` : affiche les LABEL reliant l'image au Registry MLflow.
- `docker stop scoring` : arrête le conteneur.
- `docker run --rm -i hadolint/hadolint < packaging/Dockerfile` : lint le Dockerfile (versions épinglées, cache pip).

5. Benchmark et test de charge

- `uv run python packaging/app/benchmark.py` : compare scikit-learn et ONNX Runtime (P50/P95/P99) et écrit `reports/benchmark.md`.
- `docker run -d --rm -p 8000:8000 --name scoring scoring-api:optimized` : relance le conteneur pour le test de charge.
- `uv run locust -f packaging/locustfile.py --host http://localhost:8000 --users 100 --spawn-rate 10 --run-time 30s --headless` : lance 100 utilisateurs simulés en mode headless.
- `docker stop scoring` : arrête le conteneur.

6. Variante BentoML

- `uv run python packaging/bento/save_model.py` : enregistre le modèle (même version que l'image FastAPI + contrat d'entrée) dans le model store BentoML.
- `cd packaging/bento` puis `uv run bentoml build` : construit le Bento (`scoring_bento`).
- `uv run bentoml serve scoring_bento:latest --port 3000 &` puis `sleep 10` : sert le Bento sans Docker, sur le port 3000.
- `curl -s -X POST http://localhost:3000/predict ... -d @/tmp/request.json` puis `kill %1` : teste la même requête (réponse identique à 10^{-7} près) puis arrête le service.
- `uv run bentoml containerize scoring_bento:latest -t scoring-api:bentoml` : produit l'image Docker BentoML.
- `cd ../..` puis `docker images scoring-api --format "table {{.Repository}}:{{.Tag}}\t{{.Size}}"` : liste les trois images (`naive`, `optimized`, `bentoml`).

7. Scan sécurité (Trivy + ModelScan)

- `trivy() { docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v trivy-cache:/root/.cache/ aquasec/trivy:0.75.0 "$@"; }` : lance Trivy en conteneur (version épinglée, socket Docker monté, cache de base de vulnérabilités persistant).
- `trivy image --severity HIGH,CRITICAL scoring-api:optimized > reports/trivy_optimized.txt` : scanne l'image optimisée (idem pour `naive` et `bentoml`).
- `grep -c "CVE-" reports/trivy_*.txt` et `grep "^Total" reports/trivy_*.txt` : comptent les CVE et lisent les totaux par cible.
- `uv run modelscan -p packaging/model/model.onnx > reports/modelscan_onnx.txt` puis `cat` : scanne l'ONNX (`No issues found!`).
- `uv run modelscan -p packaging/model/model.onnx --show-skipped | tail -3` : montre que l'ONNX est ignoré (format non lu).
- `uv run python packaging/scan-demo/make_pickles.py` : fabrique `model.pkl` (modèle sérialisé) et `malicious.pkl` (charge cachée inoffensive).
- `uv run modelscan -p packaging/scan-demo/ > reports/modelscan_pickle.txt` puis `grep -A4 -- "--- CRITICAL ---" reports/modelscan_pickle.txt` : détecte l'opérateur `system` dans le pickle piégé.
- `uv run mlflow artifacts download -u models:/scoring-model@champion -d /tmp/champion` puis `ls /tmp/champion` : télécharge les artefacts du champion (`model.skops`).
- `uv run modelscan -p /tmp/champion` : montre que ModelScan ignore skops.
- `uv run python -c "from skops.io import get_untrusted_types; print(get_untrusted_types(file='/tmp/champion/model.skops'))"` : liste les types hors liste de confiance (`['numpy.dtype']`, autorisé au Lab 1).

8. Push sur le registry local + test pull

- `docker ps --filter name=kind-registry --format "{{.Names}} {{.Image}} {{.Ports}}"` et `curl -s http://localhost:5001/v2/_catalog` : confirment que le registry de cours tourne déjà (`registry:3` sur `localhost:5001`).
- `docker tag scoring-api:optimized localhost:5001/scoring-api:1.0.0` : tague l'image pour le registry local.
- `docker push localhost:5001/scoring-api:1.0.0` : pousse l'image.

- `curl -s http://localhost:5001/v2/_catalog` et `curl -s http://localhost:5001/v2/scoring-api/tags/list` : vérifient la présence du dépôt et du tag `1.0.0`.
- `docker rmi scoring-api:optimized localhost:5001/scoring-api:1.0.0` puis `docker pull localhost:5001/scoring-api:1.0.0` : simule un autre environnement (suppression locale + pull).
- `docker run -d --rm -p 8000:8000 --name scoring-from-reg localhost:5001/scoring-api:1.0.0` puis `sleep 5 && curl -s http://localhost:8000/health` et `docker stop scoring-from-reg` : prouve que l'image pullée fonctionne à l'identique.
- `kubectl run scoring-smoke --image=localhost:5001/scoring-api:1.0.0 --restart=Never` puis `kubectl wait --for=condition=Ready pod/scoring-smoke --timeout=90s` : vérifie que le cluster tire l'image depuis le registry local.
- `kubectl logs scoring-smoke | tail -2` puis `kubectl delete pod scoring-smoke` : lit les logs uvicorn puis nettoie le pod.

9. Rapport et commit

- `cp labs/m4-packaging/starter/rapport-m4.md reports/rapport-m4.md` : part du modèle de rapport à compléter (5 sections).
- `git add packaging reports ':(exclude)packaging/scan-demo/*.pkl'` : indexe le travail sans les pickles de démonstration.
- `git status` : vérifie qu'aucun `.pkl` ni `.env` n'est indexé.
- `git commit -m "Lab 4: packaging du modèle"`, `git push -u origin lab/m4-packaging`, `gh pr create` : commite, pousse et ouvre la PR (relecture par le partenaire du binôme).

Pièges / points d'attention

- `skl2onnx: unsupported operator` / `MissingShapeCalculator` : le classifieur n'est pas supporté (CustomTransformer, XGBoost, LightGBM) ; rester sur un classifieur scikit-learn ou fournir un converter (`onnxmltools`).
- `NotImplementedError` sur `SimpleImputer` texte avec NaN : l'imputer de chaînes n'accepte pas NaN ; convertir via `onnx_ready()` (point (a) : `missing_values = ""`).
- `Divergence detected!` (ONNX ≠ sklearn) : entrées numériques en float32 — les arbres changent de branche ; utiliser `DoubleTensorType` + point (b) de `onnx_ready()` et passer des `float64`.
- `INVALID_ARGUMENT ... expected: (tensor(double))` : données envoyées en float32 à un graphe float64 ; convertir en `np.float64`.
- `Registered model alias champion not found` : Lab 1 non terminé ou port-forward MLflow arrêté ; relancer `make pf` et vérifier l'alias dans l'UI.
- `ModuleNotFoundError: No module named 'src'` : API lancée avec `uvicorn` au lieu de `python -m uvicorn`, ou hors de la racine ; lancer depuis la racine avec `--app-dir packaging`.
- `ImportError` sur `packaging.version` / pip / MLflow cassés : création accidentelle de `packaging/__init__.py` qui masque le paquet PyPI ; le supprimer.
- Ne créez jamais `packaging/__init__.py` : le dossier porterait le nom du paquet PyPI `packaging` et masquerait les imports.
- `docker build: no space left` : cache Docker trop gros ; `docker system prune -af --volumes` (attention : supprime aussi le volume `trivy-cache`).
- `model.onnx not found` au `COPY` : modèle non exporté ou build lancé depuis `packaging/` ; exporter puis builder depuis la racine.

- Image bloquée sur `pip install` : pas d'accès réseau au build ; vérifier le proxy Docker Desktop, pré-télécharger `python:3.12-slim`.
- `uvicorn: address already in use` : port 8000 occupé (server ou conteneur actif) ; `lsof -i:8000` + kill, ou changer de port.
- Trivy `unable to connect to Docker` : socket Docker non monté ; ajouter `-v /var/run/docker.sock:/var/run/docker.sock`.
- Trivy `TOOMANYREQUESTS` / base très lente : limite de débit ; garder `trivy-cache`, puis `--skip-db-update`.
- BentoML `model not found` : `save_model.py` jamais exécuté ; le relancer avant `bentoml build`.
- BentoML `AttributeError` sur `bentoml.Service`/`bentoml.io` : code écrit pour BentoML 1.2 ; utiliser `@bentoml.service` + `@bentoml.api`.
- `bentoml containerize : buildx introuvable` : plugin Docker `buildx` absent ; installer `docker-buildx-plugin`.
- `registry:2 : port is already allocated` : le registry du cours occupe déjà 5001 ; ne pas lancer de second registry, utiliser `kind-registry`.
- `docker push : connection refused` sur `localhost:5001` : conteneur `kind-registry` arrêté ; `docker start kind-registry` ou `make cluster`.
- Registry push `server gave HTTP response to HTTPS client` : registry désigné par un autre nom que `localhost` ; utiliser `localhost:5001` (sinon `insecure-registries`).
- `Container exit code 137` : OOM killer (RAM insuffisante) ; augmenter la RAM Docker Desktop.
- `FileNotFoundError` sur `model.onnx`/`dataset.yaml` au démarrage : fichier absent de l'image ; vérifier les `COPY` et la liste blanche du `.dockerignore`.
- ModelScan `argument of type 'NoneType' is not iterable` : il ne lit pas les pickles joblib ; sérialiser avec pickle (comme `make_pickles.py`).
- ModelScan `skipped` / code retour 3 : format non pris en charge (ONNX, skops) ; ce n'est pas une erreur.
- Ne chargez jamais `malicious.pkl` : `pickle.load`/`joblib.load` exécuteraient la charge au chargement ; ne pas commiter les `.pkl`.
- En prod, préférer ONNX (format statique, sans code) : charger un pickle revient à exécuter du code Python arbitraire ; jamais de pickle d'origine non maîtrisée.

Fiche générée à partir de Lab4-Enonce.pdf et Lab4-Commandes.pdf — cours MLOps.

Lab 5 — Déployer le modèle de scoring sur KServe · outils & commandes

Ce lab fait passer l'image Docker du Lab 4 (`localhost:5001/scoring-api:1.0.0`) à un service production-ready : installer KServe dans le cluster kind du cours (sans le recréer), exposer l'InferenceService `scoring-model` via la passerelle Kourier (port `30080`), configurer l'autoscaling (scale-to-zero → 5 pods), benchmarker avec Locust (100/500/1000 utilisateurs) et déployer une v2.0.0 en canary 90/10. Livrable : ISVC déployé + benchmark P50/P95/P99 + canary 90/10.

Outils de la stack

Outil/service	Rôle / utilité
KServe (`v0.21.0`, ns `kserve`)	CRD `InferenceService`, contrôleur et runtimes de modèles (sklearn, Triton…) : sert un modèle à partir d'un seul YAML.
Knative Serving (`knative-serving`)	Révisions immuables (image + configuration), routage du trafic entre versions, autoscaler KPA et scale-to-zero.
Kourier (`kourier-system`)	Passerelle HTTP Envoy de Knative, exposée en NodePort `30080` ; le routage se fait par l'en-tête `Host`.
cert-manager (`cert-manager`)	Émet les certificats TLS des webhooks de KServe.
metrics-server (`kube-system`)	Alimente `kubectl top` (CPU / mémoire des pods) ; installé avec `--kubelet-insecure-tls` (requis sur kind).
KEDA (`v2.21.0`, ns `keda`, option)	Autoscaling événementiel (files, Prometheus, cron) ; s'installe avec `KSERVE_WITH_KEDA=1 make kserve`.
kind	Cluster Kubernetes local du cours ; le port `30080` y est mappé depuis le Lab 0, précisément pour la passerelle KServe.
Registry local `localhost:5001`	Registry du cours branché au cluster : les nœuds tirent eux-mêmes les images (`localhost:5001/...`), plus de `kind load`.
Locust `2.46.6`	Générateur de charge : mesure le débit (RPS) et les latences P50/P95/P99 ; fait partie du socle `uv sync`.
Triton (`kserve-tritonserver`)	Serveur standard d'inférence, protocole V2, modèle ONNX téléchargé depuis S3 (comparaison de l'étape 8).
BentoML	Image HTTP du Lab 4 (pipeline scikit-learn, port `3000`) servie telle quelle par KServe.
SeaweedFS (S3)	Stockage objet du cours (`s3://models/triton` pour Triton), filer visible sur `http://localhost:8888/buckets/`.
MLflow	Registry du modèle : alias `champion` et paramètre `decision_threshold` = seuil métier servi partout.
uv	Exécution des scripts Python du repo dans un environnement synchronisé.

Cibles Makefile et scripts

Cible / script	Rôle
`make pf`	Lance les port-forwards (dont l'accès aux services) : à vérifier en début de séance.
`make kserve`	Installe Knative, Kourier, cert-manager, KServe et metrics-server via `infra/kserve/install.sh`, versions lues dans `infra/versions.env`. Idempotent : relançable en cas d'erreur réseau.
`make check-m5`	Vérification automatique non destructive (infra, service, seuil, autoscaling, canary, livrables) ; code retour = nombre de contrôles en échec.
`infra/kserve/install.sh`	Enchaîne l'installation ; configure Knative (ingress-class kourier, domaine `example.com`, NodePort 30080), exclut `localhost:5001` de la résolution des tags et patche `inferenceservice-config` (mode Knative).
`serving/monitor.sh`	Tableau de bord « live » de l'autoscaling (ISVC, KPA, pods, CPU/mémoire) rafraîchi toutes les 2 s.
`serving/payloads.py`	Construit les corps de requête V1 (`instances`) et V2 (tenseurs) depuis `dataset.yaml` et de vraies lignes de `test.csv`.
`serving/locustfile.py`	Scénario Locust : rejoue des requêtes de scoring `/predict` (ou V2 Triton selon `API_FORMAT`).
`serving/locust_report.py`	Agrège les CSV Locust en un tableau Markdown (RPS, P50/P95/P99, erreurs).
`serving/prepare_triton.py`	Dépose le graphe ONNX et `config.pbtxt` sur S3 au format attendu par Triton.
`serving/v2_client.py`	Interroge Triton en protocole V2 et décide au seuil métier publié ; `--compare` rejoue les mêmes demandes vers l'API du Lab 4.
`packaging/sample_request.py`	Génère un corps de requête de test à partir de vraies demandes (script du Lab 4).
`labs/m5-serving/check.sh`	Script exécuté par `make check-m5`.

Commandes clés, expliquées

Étape 1 — Installer KServe dans le cluster du cours

- `cd ~/mlops-cours-2026` — se placer à la racine du fork.
- `git switch main && git pull` puis `git switch -c lab/m5-serving` — créer la branche de travail du lab depuis `main` à jour.
- `set -a && source .env && set +a` — charger les variables d'environnement (S3, registry…) dans le shell.
- `make pf` — démarrer les port-forwards nécessaires.
- `kubectl config current-context` / `kubectl get nodes` — confirmer le contexte `kind-mlops-lab` et les 3 nœuds Ready.
- `docker port mlops-lab-control-plane` — vérifier le mapping `30080/tcp -> 0.0.0.0:30080`.
- `make kserve` — installer Knative, Kourier, cert-manager, KServe et metrics-server en une commande.

- `kubectl get pods -n knative-serving` (idem `kourier-system`, `cert-manager`, `kserve`) — vérifier que chaque brique tourne.
- `kubectl get crd | grep serving.kserve.io` et `kubectl get clusterservingruntimes` — vérifier les CRD et les runtimes KServe.
- `kubectl get svc kourier -n kourier-system` — confirmer le service NodePort `80:30080/TCP`.

Étape 2 — Rendre l'image disponible pour le cluster

- `curl -s http://localhost:5001/v2/scoring-api/tags/list` — confirmer que le tag `1.0.0` est bien publié dans le registry (plus besoin de `kind load`).
- `kubectl create namespace scoring` — créer le namespace qui hébergera le service.
- `mkdir -p serving reports/m5` — créer les dossiers des manifests et des rapports.

Étape 3 — Créer l'InferenceService minimal

- `kubectl apply -f serving/scoring-isvc-v1.yaml` — déployer l'ISVC (conteneur personnalisé `kserve-container`, image `1.0.0`, mode Knative).
- `kubectl get isvc -n scoring -w` — suivre jusqu'à `READY=True` (Ctrl-C ensuite).
- `kubectl get ksvc,revisions,kpa -n scoring` — inspecter le Knative Service, la révision et l'autoscaler créés pour vous.
- `kubectl get pods -n scoring` — vérifier le pod `2/2` (`kserve-container` + `queue-proxy`).
- `kubectl describe isvc scoring-model -n scoring | tail -15` — lire l'état détaillé et les conditions.

Étape 4 — Tester via la passerelle (port 30080) + curl

- `ISVC_HOST=$(kubectl get isvc scoring-model -n scoring -o jsonpath='{.status.url}' | cut -d/ -f3)` — extraire le nom d'hôte (domaine `example.com` sans DNS réel).
- `curl -s -H "Host: ${ISVC_HOST}" http://localhost:30080/health` — appeler le service via Kourier ; sans en-tête `Host`, Kourier répond 404.
- `uv run python packaging/sample_request.py -n 2 > /tmp/request.json` — fabriquer un corps de requête à partir de vraies lignes de `test.csv`.
- `curl -s -X POST http://localhost:30080/predict -H "Host: ${ISVC_HOST}" -H "Content-Type: application/json" -d @/tmp/request.json` — tester une prédiction et vérifier le seuil `0.3182`.
- `uv run python -c "... MlflowClient ... get_model_version_by_alias('scoring-model','champion') ... decision_threshold ..."` — vérifier le seuil du champion dans MLflow.
- `curl -s -H "Host: ${ISVC_HOST}" http://localhost:30080/metadata | python3 -m json.tool | grep -E '"threshold"|"model_version"'` — comparer le seuil et la version servis à ceux du champion.
- `kubectl run -n scoring client-test --rm -i --restart=Never --image=python:3.12-slim -- python -c "... urlopen('http://scoring-model-predictor.scoring/health') ..."` — appeler le service par son adresse interne (sans le port 30080).

Étape 5 — Configurer l'autoscaling

- `kubectl apply -f serving/scoring-isvc-v1-autoscale.yaml` — passer à `minReplicas: 0`, `maxReplicas: 5`, `scaleMetric: rps`, `scaleTarget: 200`.

- `kubectl get isvc scoring-model -n scoring` — voir la nouvelle révision prendre 100 % du trafic.
- `kubectl get kpa -n scoring` — voir les autoscalers et l'arrêt des pods de l'ancienne révision.
- `kubectl get revision scoring-model-predictor-00002 -n scoring -o jsonpath='{.metadata.annotations}' | python3 -m json.tool | grep autoscaling` — vérifier les annotations KPA (metric, target, min-scale, max-scale, window).
- `chmod +x serving/monitor.sh && ./serving/monitor.sh` — lancer le tableau de bord dans un terminal dédié.
- `kubectl get pods -n scoring -w` — observer le scale-to-zero (pod qui passe à `Terminating` puis disparaît).
- `curl -s -o /dev/null -w "HTTP %{http_code} en %{time_total} s\n" -H "Host: ${ISVC_HOST}" http://localhost:30080/health` — lancer deux fois : mesurer le démarrage à froid puis l'appel chaud.
- `kubectl rollout restart -n kube-system deploy/metrics-server` — corriger un `Metrics API not available` de `kubectl top`.

Étape 6 — Test de charge avec Locust

- `export ISVC_HOST` — exporter le nom d'hôte que Locust placera dans l'en-tête `Host`.
- `uv run locust -f serving/locustfile.py --host http://localhost:30080 --users 100 --spawn-rate 20 --run-time 2m --headless --csv=reports/m5/run_100 --html=reports/m5/run_100.html` — run 1 (répéter avec 500 puis 1000 utilisateurs) ; produit les rapports.
- `uv run python serving/locust_report.py` — générer le tableau Markdown des trois runs (requêtes, RPS, P50, P95, P99, erreurs).
- `BATCH_SIZE=50 uv run locust -f serving/locustfile.py --host http://localhost:30080 --users 20 --spawn-rate 20 --run-time 1m --headless --csv=reports/m5/batch_50` — bonus « batch » : 50 demandes par requête.
- `uv run python serving/locust_report.py "reports/m5/batch_*"` — résumer le run batch (comparer les prédictions/s).

Étape 7 — Déployer v2.0.0 en canary 90/10

- `MODEL_VERSION=$(uv run python -c "... metadata.json['model_version'] ...")` — récupérer la version du modèle à embarquer.
- `docker build -f packaging/Dockerfile -t localhost:5001/scoring-api:2.0.0 --build-arg VERSION=2.0.0 --build-arg MODEL_VERSION="$MODEL_VERSION" .` — construire l'image v2 (avec la route `/version`).
- `docker push localhost:5001/scoring-api:2.0.0` — publier l'image dans le registry du cours.
- `docker run -d --rm -p 8000:8000 --name scoring-v2 localhost:5001/scoring-api:2.0.0` puis `sleep 5 && curl -s http://localhost:8000/version` puis `docker stop scoring-v2` — tester l'image en local avant déploiement.
- `kubectl apply -f serving/scoring-isvc-canary.yaml` — déployer le canary (`canaryTrafficPercent: 10`, `enable-tag-routing: true`).
- `kubectl get isvc scoring-model -n scoring -w` — vérifier `PREV 90 / LATEST 10`.
- `for i in $(seq 1 200); do curl -s -H "Host: ${ISVC_HOST}" http://localhost:30080/metadata | grep -oE '"api_version":"[^"]+"'; done | sort | uniq -c` — compter la répartition réelle par version.

- `curl -s -H "Host: latest-${ISVC_HOST}" http://localhost:30080/version` et `curl -s -H "Host: prev-${ISVC_HOST}" http://localhost:30080/metadata | grep -oE '"api_version":"[^"]+"'` — interroger chaque version par son adresse dédiée.
- `kubectl patch isvc scoring-model -n scoring --type merge -p '{"spec":{"predictor":{"canaryTrafficPercent":100}}}'` — promouvoir la v2 à 100 % ; avec `0` à la place, rollback vers la v1.

Étape 8 — Triton et BentoML (optionnel)

- `uv run python serving/prepare_triton.py` — déposer le graphe ONNX et `config.pbtxt` sur S3 au format Triton.
- `kubectl create secret generic s3-credentials -n scoring --from-literal=AWS_ACCESS_KEY_ID="$AWS_ACCESS_KEY_ID" --from-literal=AWS_SECRET_ACCESS_KEY="$AWS_SECRET_ACCESS_KEY" --dry-run=client -o yaml | kubectl apply -f -` — créer le Secret S3 depuis `.env` (jamais commité).
- `kubectl annotate secret s3-credentials -n scoring --overwrite serving.kserve.io/s3-endpoint=s3.storage:8333 serving.kserve.io/s3-usehttps=0 serving.kserve.io/s3-region="$AWS_DEFAULT_REGION"` — indiquer le point d'accès SeaweedFS.
- `kubectl apply -f serving/kserve-s3-sa.yaml` — créer le ServiceAccount `kserve-s3` qui rattache le Secret aux ISVC.
- `kubectl apply -f serving/scoring-triton-isvc.yaml` puis `kubectl get isvc scoring-triton -n scoring -w` — déployer Triton (1er démarrage long).
- `TRITON_HOST=$(kubectl get isvc scoring-triton -n scoring -o jsonpath='{.status.url}' | cut -d/ -f3)` puis `curl -s -H "Host: ${TRITON_HOST}" http://localhost:30080/v2/models/scoring-model | python3 -m json.tool | head -20` — lire les métadonnées du modèle en protocole V2.
- `uv run python serving/v2_client.py --host "$TRITON_HOST" -n 5 --compare "$ISVC_HOST"` — décider au seuil publié et comparer aux prédictions de l'API du Lab 4.
- `docker tag scoring-api:bentoml localhost:5001/scoring-bento:1.0.0` puis `docker push ...` — publier l'image BentoML du Lab 4.
- `kubectl apply -f serving/scoring-bento-isvc.yaml` puis `kubectl wait --for=condition=Ready isvc/scoring-bento -n scoring --timeout=300s` — déployer et attendre la disponibilité.
- `BENTO_HOST=$(kubectl get isvc scoring-bento -n scoring -o jsonpath='{.status.url}' | cut -d/ -f3)` puis `curl -s -X POST http://localhost:30080/predict -H "Host: ${BENTO_HOST}" -H "Content-Type: application/json" -d @/tmp/request.json` — tester BentoML.
- `ISVC_HOST=$BENTO_HOST API_FORMAT=v1 uv run locust -f serving/locustfile.py --host http://localhost:30080 --users 20 --spawn-rate 20 --run-time 1m --headless --only-summary --csv=reports/m5/compare_bento` — même banc d'essai pour les trois serveurs (répéter pour `compare_fastapi` en v1 et `compare_triton` en v2).
- `uv run python serving/locust_report.py "reports/m5/compare_*"` — comparer les trois serveurs.
- `kubectl delete -f serving/scoring-triton-isvc.yaml -f serving/scoring-bento-isvc.yaml` — supprimer les variantes après la mesure.

Étape 9 — Rédiger le rapport et livrer

- `cp labs/m5-serving/starter/rapport-m5.md reports/rapport-m5.md` — partir du modèle de rapport fourni.
- `git add serving packaging/app/serve.py packaging/Dockerfile reports/rapport-m5.md reports/m5/*_stats.csv` — indexer les livrables (les HTML Locust restent sur le poste).
- `git commit -m "Lab 5: serving KServe (autoscaling, canary, benchmark)"` — commiter.
- `git push -u origin lab/m5-serving` puis `gh pr create` — pousser et ouvrir la pull request.
- `make check-m5` — lancer la vérification automatique finale.

Pièges / points d'attention

- `make kserve` : erreur x509 / connection refused sur un webhook — le webhook n'est pas encore prêt ; le script réessaie seul, sinon relancer `make kserve` (idempotent).
- ISVC `READY=False` avec pod en `ImagePullBackOff` — image absente du registry ou nom erroné ; vérifier `curl localhost:5001/v2/scoring-api/tags/list`. Pas de `kind load` : le registry est branché au cluster.
- `failed to resolve image to digest` / `RevisionMissing` — Knative tente de résoudre le tag `localhost:5001/...` ; relancer `make kserve` (exclusion `registries-skipping-tag-resolving`), puis supprimer et réappliquer l'ISVC.
- Un tag publié est immuable : ne jamais repousser une image différente sous un tag existant ; chaque nouvelle version reçoit son propre tag.
- `update rejected: deploymentMode cannot be changed` — le mode est figé à la création de l'ISVC : supprimer (`kubectl delete isvc`) puis réappliquer.
- Canary `100/0` au lieu de `90/10` — ISVC en mode Standard (où `canaryTrafficPercent` est ignoré) ou absence de révision précédente déployée à 100 %.
- `/version` renvoie `{"detail":"Not Found"}` — 9 requêtes sur 10 tombent sur la révision 1.0.0 (image sans `/version`) ; tester via l'adresse `latest-...` ou lire `/metadata`.
- Deux `model_version` différents entre MLflow et le service — image construite avant un changement de champion : réexporter le modèle et reconstruire l'image (Lab 4).
- `curl localhost:30080` : `Connection refused` — service `kourier` pas en NodePort 30080, ou cluster créé sans le mapping ; vérifier `kubectl get svc kourier -n kourier-system` et `docker port mlops-lab-control-plane`.
- `curl` : 404 avec réponse vide — en-tête `Host` absent ou erroné ; le recalculer via `jsonpath='{.status.url}'`.
- `curl /predict` : 503 — pod pas encore prêt ; `kubectl wait --for=condition=Ready pod -l serving.kserve.io/inferenceservice=scoring-model -n scoring`.
- Premier appel lent (quelques secondes) — démarrage à froid normal après scale-to-zero ; garder `minReplicas: 1` pour l'éviter.
- `kubectl top pods` : `Metrics API not available` — metrics-server pas prêt ; attendre 60 s ou `kubectl rollout restart -n kube-system deploy/metrics-server`.
- Sous charge, `DESIREDSCALE` reste à 1 — `scaleTarget` au-dessus de ce qu'un pod sait servir ; baisser la cible ou passer à `scaleMetric: concurrency`.
- Locust 100 % d'échecs / 404 — `ISVC_HOST` non exporté dans le terminal de Locust (`export ISVC_HOST=...`).
- Locust : connection reset au run 1000 — service saturé (attendu) ou Locust lui-même (`CPU usage above 90%`) ; augmenter `maxReplicas`, relâcher les limites CPU, ou ajouter `--processes 4`.

- Triton : `ProgressDeadlineExceeded` — téléchargement de l'image Triton (≈ 6 Go) trop long ; l'annotation `serving.knative.dev/progress-deadline` est déjà à 30 min.
- Triton : `storage-initializer` en erreur (`NoCredentialsError`, `EndpointConnectionError`) — Secret absent/mal annoté ou ServiceAccount non référencé ; revoir l'étape 8.2 et `serviceAccountName: kserve-s3`.
- Triton : modèle `UNAVAILABLE`, failed to load — mauvaise arborescence : `storageUri` doit désigner le dossier contenant `scoring-model/`.
- Triton : 400 `unexpected datatype` — type d'entrée différent du graphe ONNX ; construire la requête avec `serving/payloads.py` (`v2_body`).
- `prepare_triton.py` : `XAmzContentSHA256Mismatch` — export `AWS_REQUEST_CHECKSUM_CALCULATION=when_required` puis relancer.
- Kourier : `SETTINGS_MAX_CONCURRENT_STREAMS` (connexion gRPC périmée) — `kubectl rollout restart deploy/3scale-kourier-gateway -n kourier-system`.
- Passerelle indisponible — port-forward de secours : `kubectl port-forward -n scoring deploy/scoring-model-predictor-00001-deployment 8001:8000` (adapter le n° de révision ; le port 8080 est pris par l'UI KFP).
- Nœud kind plein (`no space left`) — images accumulées : `docker exec mlops-lab-worker crictl rmi --prune` (idem `mlops-lab-worker2` et `mlops-lab-control-plane`).
- Ressources : prévoir ≥ 8 Go de RAM pour Docker ; Knative + Kourier + cert-manager + KServe ≈ 1,5 Go. Libérer Airflow (Lab 2) et mettre KFP en pause plutôt que recréer le cluster.
- La cible de l'autoscaler (`scaleTarget`) dépend du coût d'une prédiction : un modèle plus lourd demandera une cible plus basse ; un gros modèle allonge aussi le démarrage à froid du scale-to-zero.
- Le protocole V2 de Triton ne transporte pas de valeur manquante numérique en JSON : le client V2 écarte ces lignes (elles restent servies par l'API du Lab 4 qui accepte `null`).

Fiche générée à partir de Lab5-Enonce.pdf et Lab5-Commandes.pdf — cours MLOps.

Lab 6 — CI/CD/CT de bout en bout sur K8s · outils & commandes

Ce lab ferme la boucle MLOps : une **dérive** détectée par Evidently déclenche un **ré-entraînement** (KFP), une **gate** promeut (ou refuse) le modèle dans MLflow, puis une mise en **production GitOps** (Argo CD) déploie un canary KServe — sans intervention humaine. Trois chaînes tournent en parallèle et ne communiquent que par **Git** : la CI (GitHub Actions), le CD (Argo CD dans le cluster) et le CT (webhook sur le poste).

Outils de la stack

Outil/service	Rôle / utilité
GitHub Actions	Exécute la CI sur runners hébergés : lint, tests, build de l'image, test de fumée, publication GHCR, puis commit du tag de staging.
GHCR (`ghcr.io`)	Registry d'images ; stocke `scoring-api:2.1.N`, tiré par les nœuds kind.
Git / GitHub	Source unique de vérité : toute décision de déploiement est un commit (GitOps).
Argo CD	CD GitOps : lit `k8s/overlays/*` et synchronise `scoring-staging`/`scoring-prod` depuis Git, avec `prune` et `selfHeal`.
Kustomize	Construit les manifests (une base + un overlay par environnement) qu'Argo CD applique.
KServe (`InferenceService`)	Sert le modèle ; fournit le canary natif en mode Knative.
Knative	Mode de déploiement KServe activant `canaryTrafficPercent` et le scale-to-zero.
Kubeflow Pipelines (KFP)	Exécute le pipeline de ré-entraînement `ct-pipeline`.
MLflow	Registry du modèle (alias `@champion`/`@challenger`) et rangement des lots et rapports de dérive (`scoring-drift`).
Evidently 0.7	Détecte la dérive entre les données de référence (train) et le lot récent.
Webhook CT (`trigger_ct.py`)	Reçoit l'événement de dérive, décide, lance le pipeline KFP et suit le run.
Pipelines GitOps (`gitops.py`)	Transforme une décision de déploiement en commit (staging, canary, promote, rollback).
kind	Cluster Kubernetes local du lab (K8s in Docker).
kubectl	Pilote le cluster : Applications, InferenceServices, secrets, port-forwards.
Docker	Construit l'image (test de fumée en CI ; registry local `localhost:5001`).
Registry local (`localhost:5001`)	Registry des images du cluster (prod initiale du Lab 5).
`uv`	Gère l'environnement Python (`uv.lock`) et exécute les modules du projet.
ruff	Lint et vérification de format.
pytest	Tests (contrat, API, manifests, pipeline, webhook).

Outil/service	Rôle / utilité
`gh` (CLI)	Création de PR et suivi des runs (facultatif).
Argo Rollouts	**Non utilisé** : il pilote un Deployment, pas un `InferenceService` (le canary KServe le remplace).

Cibles Makefile et scripts

Cible / script	Rôle
`make pf`	Lance les port-forwards (MLflow, etc.) nécessaires au poste.
`make kfp-ui`	Port-forward de l'UI/API KFP (port 8080) ; expose `apis/v2beta1`.
`make argocd`	Installe Argo CD sur kind via `infra/argocd/install.sh` (v3.5.3).
`make pipeline-image`	Construit/pousse l'image du pipeline (`scoring-pipeline`, Lab 2).
`make check-m6`	Vérification automatique du lab (fichiers, CI, overlays, Argo CD, CT, Git) sans rien modifier.
`infra/argocd/install.sh`	Applique le manifest Argo CD non-HA (`--server-side`), règle la réconciliation à 60 s, met Dex/notifications/ApplicationSets à 0 réplique.
`pipelines/gitops.py`	Outil GitOps : sous-commandes `status`, `payload`, `deploy`, `canary`, `promote`, `rollback`, `release` (+ `local-ci`).
`pipelines/trigger_ct.py`	Webhook CT : sous-commandes `simulate`, `detect`, `send`, `serve`.
`pipelines/ct_pipeline.py`	Pipeline KFP de ré-entraînement ; se compile avec `python -m pipelines.ct_pipeline`.
`packaging/app/export_onnx.py`	Exporte le champion MLflow en ONNX + métadonnées dans `packaging/model/`.
`labs/m6-cicd/check.sh`	Script exécuté par `make check-m6`.
`labs/m6-cicd/starter/git-server.yaml`	Git daemon en cluster (alternative locale sans GitHub Actions).
`labs/m6-cicd/starter/rapport-m6.md`	Gabarit du rapport final (copié dans `reports/`).

Commandes clés, expliquées

Étape 1 — Structurer le repo (tests, manifests)

- `git switch main && git pull` puis `git switch -c lab/m6-cicd` : partir de main à jour et créer la branche du lab.
- `uv sync` : installe l'environnement exact de `uv.lock`.
- `set -a && source .env && set +a` : exporte les variables d'environnement (dont le jeton du webhook).
- `make pf` : ouvre les port-forwards vers MLflow.
- `git ls-files packaging/model` : vérifie que le modèle du Lab 4 (`model.onnx`, `metadata.json`) est versionné.

- `mkdir -p .github/workflows k8s/base k8s/overlays/staging k8s/overlays/prod` : crée l'arborescence CI + manifests.
- `cp labs/m6-cicd/starter/gitops.py pipelines/gitops.py` : installe l'outil GitOps fourni.
- `uv run python -m pipelines.gitops --help` : liste la surface de l'outil (`status`, `payload`, `deploy`, `canary`, `promote`, `rollback`, `release`).
- `uv run ruff check . && uv run ruff format --check .` : lint et contrôle de format.
- `uv run pytest -q` : exécute la suite (24 tests attendus : contrat, API, manifests).

Étape 2 — Pipeline CI (lint + tests + build + push)

- `git add .github k8s tests pipelines/gitops.py` puis `git commit -m "..."` et `git push -u origin lab/m6-cicd` : pousse la branche.
- `gh pr create` : ouvre la pull request (déclenche la CI en test, sans publication).
- `gh pr checks --watch` : suit les checks de la PR.
- `gh run watch` : suit le run de main (publication + déploiement de staging après fusion).
- `git log --oneline -3` : montre le commit GitOps poussé par le job `deploy-staging`.
- `kubectl create namespace "$ns" --dry-run=client -o yaml | kubectl apply -f -` : crée les namespaces de façon idempotente.
- `kubectl create secret docker-registry ghcr-pull ...` : secret de tirage GHCR (uniquement si le package reste privé).
- `curl ... https://ghcr.io/v2/${REPO}/tags/list` : vérifie l'accès anonyme au package public.

Étape 3 — Installer Argo CD

- `make argocd` : installe Argo CD et règle la réconciliation.
- `kubectl port-forward -n argocd svc/argocd-server 8443:443 &` : expose l'interface web sur `https://localhost:8443`.
- `kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath=... | base64 -d` : récupère le mot de passe admin initial.
- `curl -sSL .../argocd-linux-amd64` + `sudo install ...` : installe la CLI `argocd` (même version que le serveur).
- `argocd login localhost:8443 --username admin --insecure` : connexion de la CLI.

Étape 4 — Les Applications Argo CD

- `kubectl kustomize k8s/overlays/prod` : rendu de l'overlay prod (l'`InferenceService` unique, image + canary).
- `sed -i "s#VOTRE_USER#.../#" k8s/argocd-app.yaml` : remplace le compte GitHub dans les `repoURL`.
- `kubectl apply -f k8s/argocd-app.yaml` : crée les deux Applications (`scoring-staging`, `scoring-prod`).
- `kubectl get applications -n argocd -w` : suit l'état `Synced/Healthy`.
- `kubectl get isvc -A` : liste les `InferenceService` (prod + staging).
- `kubectl get isvc scoring-model -n scoring -o jsonpath='{.metadata.annotations...,tracking-id}'` : preuve qu'Argo CD suit l'ISVC de prod.
- `kubectl get revisions -n scoring` : liste les révisions KServe.
- `kubectl patch isvc ... canaryTrafficPercent:50` puis `kubectl get isvc ... -w` : démontre le `selfHeal` (retour à 100 % en < 70 s).

Étape 5 — Déploiement par git push

- `git commit -am "..."` + `git push` + `gh pr create` : modifie l'app et ouvre la PR.
- `git switch main && git pull` : récupère la fusion et les commits des outils.
- `kubectl get applications -n argocd` : état des Applications.
- `uv run python -m pipelines.gitops status` : compare ce que Git dit de staging/prod et ce qui est réellement servi.

Étape 6 — Dérive Evidently + webhook

- `echo "CT_WEBHOOK_TOKEN=$(openssl rand -hex 16)" >> .env` : génère le jeton du webhook (jamais commité).
- `make kfp-ui` et `curl -s localhost:8080/apis/v2beta1/healthz` : ouvrent et testent l'API KFP.
- `CT_AUTO_RELEASE=0 uv run python -m pipelines.trigger_ct serve` : lance le webhook sans mise en prod automatique.
- `curl -s localhost:8095/health` : état du webhook.
- `curl -s -X POST localhost:8095/drift -H ... -d '{"drift_share": 1}'` : envoie un événement (401 sans jeton → contrôle d'accès).
- `uv run python -m pipelines.trigger_ct simulate` : fabrique un lot récent étiqueté avec dérive (`data/drift/current.csv`).
- `uv run python -m pipelines.trigger_ct detect --no-send` : produit le rapport Evidently et l'événement, sans POST.

Étape 7 — Pipeline CT (webhook → KFP → nouveau modèle)

- `uv run python -m pipelines.ct_pipeline && grep -c "^ exec-" ct_pipeline.yaml` : compile le pipeline (7 exécuteurs attendus).
- `uv run pytest -q` : suite complète (29 tests attendus).
- `uv run python -m pipelines.trigger_ct send --event reports/drift/drift_event.json` : envoie l'événement au webhook, qui soumet le run KFP.
- `uv run python -c "from mlflow import MlflowClient; ...get_model_version_by_alias('scoring-model','champion')..."` : inspecte l'alias champion et ses tags (`promotion`, `previous_champion`, `ct_trigger`).
- `git add pipelines tests` + `git commit` + `git push` + `gh pr create` : troisième PR du lab.

Étape 8 — Canary KServe piloté par Git

- `uv run python packaging/app/export_onnx.py` : exporte le champion dans `packaging/model/`.
- `git add packaging/model && git commit -m "..." && git push` + `gh run watch` : publie le modèle comme le ferait un humain.
- `uv run python -m pipelines.gitops deploy --env prod --canary 10 --push` : un commit qui change l'image prod ET la part du canary.
- `git show --stat HEAD` : vérifie les fichiers modifiés (`kustomization.yaml`, `canary.yaml`).
- `kubectl get isvc scoring-model -n scoring` : répartition PREV/LATEST.
- `for i in $(seq 1 200); do curl -s -H "Host: ${ISVC_HOST}" .../version ...; done | sort | uniq -c` : mesure la part de trafic réelle.
- `uv run python -m pipelines.gitops promote --push` : promeut à 100 %.
- `uv run python -m pipelines.gitops canary 0 --push` : retour arrière instantané.

- `uv run python -m pipelines.gitops rollback --push` : canary 0 + restauration du `@champion` précédent.

Étape 9 — Démo de bout en bout + rapport

- `uv run python -m pipelines.trigger_ct serve` : relance le webhook en mode automatique (mise en prod par défaut).
- `uv run python -m pipelines.trigger_ct simulate --factor 2` puis `detect` : dérive plus forte, cycle complet sans intervention.
- `git log --format='%ci %an | %s' -6` : horodatages des commits (temps de chaque étape).
- `curl -s localhost:8095/runs` : historique des runs suivis par le webhook.
- `cp labs/m6-cicd/starter/rapport-m6.md reports/rapport-m6.md` : installe le gabarit de rapport.
- `git switch main && git pull && git switch -c lab/m6-rapport` + `git add reports/...` + `gh pr create` : dernière PR.
- `make check-m6` : vérification automatique finale (code retour = nombre d'échecs).

Pièges / points d'attention

- `paths` **de la CI** : un commit qui ne touche que `k8s/` ne relance rien — c'est volontaire (évite une boucle infinie).
- **GHCR en minuscules** : le nom de l'image doit être en minuscules (`${VAR,,}`), sinon le push échoue ; `packages: write` suffit via `GITHUB_TOKEN`.
- **Package GHCR privé à la création** : les nœuds kind ne peuvent pas le tirer (`ImagePullBackOff`) ; le rendre public ou créer un secret `ghcr-pull` (+ `imagePullSecrets`).
- **Tag immuable** : jamais `latest` ; un tag = une image, on ne réécrit pas une image publiée.
- **Modèle versionné dans Git** : `packaging/model/` voyage dans le repo ; au-delà de quelques dizaines de Mo, passer à Git LFS ou à un `storageUri`.
- **Dérive seulement sur le numérique continu** : `simulate` ne dérive que les variables à plus de 20 valeurs distinctes ; sinon fournir un vrai lot via `detect --current`.
- **Gate = étiquettes requises** : une gate n'a de sens que si le lot récent est étiqueté.
- `kubectl patch` **sans effet durable** : Argo CD (`selfHeal`) réapplique l'état de Git ; tout changement passe par un commit (`gitops.py`).
- `gitops --push` **depuis une branche** `lab/*` : refusé, l'outil ne pousse que depuis `main` à jour.
- **Webhook 401/503** : `CT_WEBHOOK_TOKEN` absent ou `.env` non chargé (`set -a && source .env && set +a` dans les deux terminaux).
- **Webhook 502** : port-forward KFP arrêté → relancer `make kfp-ui`.
- `skipped` **(cooldown)** : délai minimal entre deux ré-entraînements (`CT_COOLDOWN_SECONDS`), un seul run à la fois.
- `uv sync` **en CI** : `UV_LOCKED=1` fait échouer si `uv.lock` ne correspond plus à `pyproject.toml` (`uv lock` en local, commiter `uv.lock`).
- `deploy-staging` **en échec au push** : commit concurrent ou `main` protégée → relancer (rebase) ou autoriser le bot.
- **Argo CD** `Unknown / repository not found` : `repoURL` encore à `VOTRE_USER`, ou fork privé.
- **Gate REFUSÉE** : comportement normal — le challenger fait moins bien que le champion ; le champion reste en place.
- **Staging « servi : None »** : scale-to-zero (premier appel lent) ou révision en échec.

- **Sécurité du CT automatique** : une mise en prod directe est risquée ; garde-fous = gate (seuil + coût), passage par Git/CI puis par un canary à 10 %, et en réel une validation humaine (PR plutôt que push).

Fiche générée à partir de Lab6-Enonce.pdf et Lab6-Commandes.pdf — cours MLOps.

Lab 7 — Monitoring en production du modèle de scoring · outils & commandes

Ce lab installe les signaux d'alerte autour de l'API de scoring servie par KServe : un modèle qui se dégrade ne plante pas, il répond `200 OK` avec des probabilités plausibles pendant que la population change. On instrumente l'API avec Prometheus, on construit un dashboard Grafana, on détecte le drift avec Evidently, on agrège les logs avec Loki, on configure des alertes SLO et on démontre le cycle drift → alerte → ré-entraînement puis la décision de promotion d'un canary à partir des métriques.

Outils de la stack

Outil/service	Rôle / utilité
`prometheus-client`	Bibliothèque Python instrumentant l'API : compteurs, histogrammes, `Info`, route `GET /metrics`. Version 0.24.1 imposée par le `uv.lock` (Feast exige `<0.25`).
Prometheus	Base de séries temporelles : scrape l'API toutes les 15 s, stocke les métriques (4 signaux d'or + métriques métier) et évalue les règles d'alerte.
Grafana	Visualisation : dashboard versionné `scoring-overview` (SLO, modèle, drift, infra, logs) + source de données Loki.
Alertmanager	Routage des alertes : règles SLO (burn rate), drift, taux de refus ; envoi vers Slack/webhook par sévérité.
kube-prometheus-stack	Chart Helm (91.9.0) qui déploie Prometheus, Alertmanager, Grafana, operator, kube-state-metrics et node-exporter.
Pushgateway	Reçoit en `PUT` les métriques poussées par le job de drift (un CronJob vit trop peu pour être scrapé directement).
Evidently (0.7)	Calcule le drift (méthode PSI) entre les données d'entraînement et les demandes réelles, et produit un rapport HTML.
Loki	Stockage des logs, mode monolithique (un pod), rétention 72 h (les prédictions sont des données personnelles).
Grafana Alloy	Collecte des logs de tous les pods via l'API Kubernetes et les envoie à Loki (successeur de Promtail, en fin de vie depuis mars 2026).
KServe / InferenceService	Sert le modèle de scoring ; expose `scoring-model` et gère la répartition du trafic entre révisions (canary).
Knative	Sous-couche de l'InferenceService : renomme le port 8000 en `user-port`, ajoute le label `serving.knative.dev/revision` (clé pour comparer stable/canary).
DVC	Versionne la référence (`data/processed/train.csv`) : le fichier est rangé sur le S3 sous son empreinte MD5, `dvc.api.get_url` donne l'adresse sans téléchargement.
S3 (SeaweedFS)	Stockage objet du cours : référence DVC et rapports de drift HTML (`s3://monitoring/drift-reports`).
Helm	Installe les charts d'observabilité avec versions épinglées (`helm upgrade --install`, idempotent).

Outil/service	Rôle / utilité
kind / Docker Desktop	Cluster Kubernetes local ; kube-prometheus-stack ≈ 2 Go, Loki + Alloy ≈ 0,5 Go, au moins 10 Go alloués à Docker.
Argo CD / GitOps	Déploie par Git : fusion d'une PR → image GHCR → staging ; la production ne bouge que par commit de canary/promotion/rollback.
webhook.site / Slack	Récepteur de test (webhook.site) ou canal réel du webhook d'alerte Alertmanager.
KFP (Kubeflow Pipelines)	Exécute le pipeline de ré-entraînement `ct-pipeline` appelé par le webhook continu du Lab 6.

Cibles Makefile et scripts

Cible / script	Rôle
`make monitoring`	Installe/actualise toute la stack d'observabilité via Helm (idempotent : à relancer après toute modification des values).
`make pf`	Port-forwards du socle (MLflow, etc.) ; condition de départ de la séance avec `kubectl get nodes`.
`make kfp-ui`	Ouvre l'interface Kubeflow Pipelines (port 8080), nécessaire au ré-entraînement de l'étape 9.
`make check-m7`	Vérifie le lab sans rien modifier : fichiers, tests, stack prête, scraping, métriques, dashboard, alertes, drift, livrables. Code retour = nombre d'échecs.
`infra/monitoring/install.sh`	Ajoute les dépôts de charts et épingle les versions ; install de kube-prometheus-stack, Pushgateway, Loki, Alloy.
`infra/monitoring/port-forward.sh`	Expose les interfaces (Grafana 3000, Prometheus 9090, Alertmanager 9093, Loki 3100, Pushgateway 9091) ; `stop` les arrête.
`monitoring/traffic.py`	Générateur de trafic : rejoue des lignes réelles de `test.csv` à débit constant ; sait déformer les données (`--mode drift`) pour simuler un drift.
`monitoring/evidently-job/calc_drift.py`	Job de drift : référence DVC vs demandes réelles lues dans Loki, calcul PSI, push Pushgateway, rapport S3, webhook de ré-entraînement.
`monitoring/canary_analysis.py`	Compare stable et canary révision par révision (six garde-fous) et décide ROLLBACK / WAIT / palier suivant / PROMOTE ; `--apply --push` applique par Git.
`packaging/app/serve.py`	API de scoring instrumentée : `/metrics`, middleware `count_requests`, `observe()`, journal JSON des prédictions.
`packaging/sample_request.py`	Construit une charge de test (lit `test.csv` et `dataset.yaml`) ; option `-n` pour le nombre de demandes.
`monitoring/dashboards/kustomization.yaml`	Transforme les JSON versionnés en ConfigMap labellisée `grafana_dashboard: "1"` (détectée par le sidecar Grafana).
`monitoring/scoring-podmonitor.yaml`	PodMonitor ciblant les pods de l'InferenceService sur le port 8000, chemin `/metrics`.

Cible / script	Rôle
`monitoring/evidently-cronjob.yaml`	CronJob `scoring-drift`, toutes les 15 min, qui lance l'image du job de drift.
`monitoring/scoring-alerts.yaml`	`PrometheusRule` : recording rules (`scoring.rules`) + six alertes (`scoring.alerts`).
`monitoring/alertmanager-config.yaml`	`AlertmanagerConfig` : routage des alertes `ml-platform` vers Slack par sévérité (webhook lu dans un Secret).

Commandes clés, expliquées

Préparation de la branche

- `git switch main && git pull` puis `git switch -c lab/m7-monitoring` : part d'un `main` à jour et crée la branche de travail du lab.

Étape 1 — Instrumenter l'API

- `set -a && source .env && set +a` : charge les variables d'environnement (identifiants S3, tokens…) dans le shell courant.
- `uv sync` : installe/synchronise l'environnement Python, notamment `prometheus-client`.
- `uv run python -c "from importlib.metadata import version; print(version('prometheus-client'))"` : vérifie la version installée (attendu `0.24.1`).
- `uv run python -m uvicorn app.serve:app --app-dir packaging --port 8000 > /tmp/api.log 2>&1 &` : lance l'API en arrière-plan, logs dans `/tmp/api.log`.
- `uv run python packaging/sample_request.py -n 20 > /tmp/request.json` : génère une charge de 20 demandes réelles.
- `for i in $(seq 1 20); do curl -s -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d @/tmp/request.json > /dev/null; done` : envoie 20 × 20 requêtes à `/predict` pour alimenter les métriques.
- `curl -s http://localhost:8000/metrics | grep -E "^(ml_|http_requests)" | grep -v _bucket` : lit les métriques métier et HTTP exposées (hors `_bucket`).
- `grep -m1 '"event":"prediction"' /tmp/api.log` : vérifie qu'une ligne JSON de prédiction est bien journalisée.
- `kill %1` : arrête l'API lancée en arrière-plan.
- `cp -r labs/m7-monitoring/starter/monitoring .` et `cp labs/m7-monitoring/starter/tests/test_monitoring.py tests/` : copie les fichiers fournis (tests, trafic, canary, dashboard, runbooks).
- `uv run pytest tests/test_monitoring.py -v` : lance les tests du lab (attendu `13 passed`) ; ils tournent aussi dans la CI, bloquant une régression d'observabilité.

Étape 2 — Déployer par GitOps

- `git add …` / `git commit -m "feat(api): …"` / `git push -u origin lab/m7-monitoring` : pousse l'API instrumentée sur la branche.
- `gh pr create` puis `gh pr checks --watch` : ouvre la PR et suit la CI (lint, tests, build de l'image).
- `gh run watch` : suit le run de `main` (publication de l'image + déploiement staging).
- `uv run python -m pipelines.gitops status` : affiche l'état staging/prod (nouveau tag en staging, ancien en prod).

- `uv run python -m pipelines.gitops deploy --env prod --canary 10 --push` : premier canary en production, commité par l'outil GitOps (part de `main`).
- `kubectl get isvc scoring-model -n scoring -w` : suit l'InferenceService jusqu'à `READY=True` et `LATEST=10` (Ctrl-C).
- `ISVC_HOST=$(kubectl get isvc scoring-model -n scoring -o jsonpath='{.status.url}' | cut -d/ -f3)` puis `export ISVC_HOST` : extrait l'hôte de la passerelle KServe pour les requêtes avec `Host:`.
- `curl -s -H "Host: latest-${ISVC_HOST}" http://localhost:30080/metrics | grep ^ml_model_info` : vérifie que la nouvelle révision expose `ml_model_info` (l'ancienne répond 404 : non instrumentée).
- `uv run python -m pipelines.gitops promote --push` : promeut le canary à 100 % une fois le contrôle de fumée passé.

Étape 3 — Installer la stack

- `make monitoring` : installe kube-prometheus-stack, Pushgateway, Loki et Alloy (relancer après toute modification des values).
- `kubectl get pods -n monitoring` : attend que tous les pods soient `Running`/prêts (3 à 5 min).
- `./infra/monitoring/port-forward.sh` : ouvre les interfaces locales (Grafana `admin`/`admin`, Prometheus, Alertmanager, Loki, Pushgateway).

Étape 4 — PodMonitor

- `kubectl apply -f monitoring/scoring-podmonitor.yaml` : installe le PodMonitor ciblant les pods de l'ISVC (nécessaire car en Knative les Services n'exposent que le queue-proxy).
- `kubectl get podmonitor -n monitoring` : confirme sa présence.
- Requêtes Prometheus de contrôle : `up{namespace="scoring", isvc="scoring-model"}`, `ml_model_info{namespace="scoring"}`, `sum by (outcome) (ml_predictions_total{namespace="scoring"})`.

Étape 5 — Trafic et dashboard

- `uv run python monitoring/traffic.py --url http://localhost:30080 --host-header "$ISVC_HOST" --duration 0` : trafic normal illimité (à laisser tourner jusqu'à l'étape 9).
- `kubectl apply -k monitoring/dashboards` : charge le dashboard versionné via la ConfigMap labellisée.
- `kubectl get configmap -n monitoring -l grafana_dashboard=1 | grep scoring` : vérifie que la ConfigMap `scoring-dashboards` existe.

Étape 6 — CronJob Evidently

- `docker build -f monitoring/evidently-job/Dockerfile -t localhost:5001/scoring-drift:1.0.0 .` puis `docker push localhost:5001/scoring-drift:1.0.0` : construit et publie l'image du job de drift (contexte = racine du repo).
- `curl -s http://localhost:5001/v2/scoring-drift/tags/list` : vérifie le tag publié.
- `kubectl create secret generic s3-credentials -n monitoring --from-literal=… --dry-run=client -o yaml | kubectl apply -f -` : crée le Secret d'accès S3 dans `monitoring` (idempotent).
- `dvc push` : pousse la référence sur le S3 (sinon `NoSuchKey`).

- `REFERENCE_URI=$(uv run python -c "import dvc.api; print(dvc.api.get_url('data/processed/train.csv'))")` puis ConfigMap `scoring-drift-reference` : donne au job l'adresse de la référence versionnée.
- `kubectl apply -f monitoring/evidently-cronjob.yaml` : déploie le CronJob `scoring-drift` (`*/15 * * * *`).
- `kubectl create job --from=cronjob/scoring-drift manual-run-1 -n monitoring` puis `kubectl logs -n monitoring job/manual-run-1 -f` : déclenche un run immédiat et suit ses logs.
- `aws-lab s3 ls s3://monitoring/drift-reports/` et `aws-lab s3 cp s3://monitoring/drift-reports/<fichier>.html - > /tmp/drift.html` : liste et récupère le rapport HTML via la sortie standard.

Étape 7 — Loki + Alloy

- Requêtes LogQL d'exploration : `{namespace="scoring", container="kserve-container"}`, le filtre `|= \"event":"prediction"` ` | json | prediction = "1"`(refus) etsum by (revision) (count_over_time(… [1m]))` (comptage par révision).

Étape 8 — Alertes et Slack

- `kubectl apply -f monitoring/scoring-alerts.yaml` : charge les recording rules et les six alertes.
- `kubectl create secret generic slack-webhook -n monitoring --from-literal=url="…" --dry-run=client -o yaml | kubectl apply -f -` : stocke l'URL du webhook (secret, jamais dans Git).
- `kubectl apply -f monitoring/alertmanager-config.yaml` : configure le routage par sévérité.
- `curl -s http://localhost:9093/api/v2/status | grep -o 'monitoring/scoring-slack/slack-ml[a-z-]*' | sort -u` : vérifie que les receivers préfixés sont enregistrés.
- `curl -s -X POST http://localhost:9093/api/v2/alerts -H "Content-Type: application/json" -d '[{…}]'` : injecte une alerte de test pour valider la chaîne sans attendre une panne.

Étape 9 — Démo drift → alerte → ré-entraînement

- `uv run python monitoring/traffic.py --url http://localhost:30080 --host-header "$ISVC_HOST" --mode drift --drift age=0.6 --drift credit_amount=2.5 --duration 1200` : injecte un drift (âges × 0,6, montants × 2,5).
- `kubectl create job --from=cronjob/scoring-drift manual-run-2 -n monitoring` puis `kubectl logs -n monitoring job/manual-run-2 -f` : relance l'analyse de drift sans attendre le quart d'heure.
- `CT_AUTO_PROMOTE=0 uv run python -m pipelines.trigger_ct serve` : lance le webhook de ré-entraînement du Lab 6 sur `localhost:8095` (sans auto-promotion : il s'arrête au canary à 10 %).
- `REFERENCE_URI=… LOKI_URL=http://localhost:3100 PUSHGATEWAY_URL="" REPORTS_URI="" CT_WEBHOOK_URL=http://localhost:8095/drift uv run python monitoring/evidently-job/calc_drift.py` : rejoue le calcul depuis le poste en pointant le webhook ; attendu un HTTP 202 avec un `run_id`.
- `cp labs/m7-monitoring/starter/rapport-m7.md reports/rapport-m7.md` : crée le squelette du rapport (six sections).

Étape 10 — Analyser le canary

- `kubectl get isvc scoring-model -n scoring` : affiche les colonnes PREV/LATEST (ex. 90/10).
- `uv run python monitoring/traffic.py --url http://localhost:30080 --host-header "$ISVC_HOST" --duration 180` : trafic normal pour juger le canary.
- `uv run python monitoring/canary_analysis.py` : analyse les six garde-fous et propose une décision (CONTINUE/ROLLBACK/PROMOTE).
- `uv run python monitoring/canary_analysis.py --apply --push` : applique la décision par Git (canary 50 %, puis promotion 100 %).
- `git log --oneline -3` : vérifie les commits `gitops(prod): …`.
- `uv run python -m pipelines.gitops deploy --env prod --canary 10 --image localhost:5001/scoring-api --tag 2.0.0 --push` : simule une release défectueuse (image sans métriques) → attendu ROLLBACK.
- `uv run python -m pipelines.gitops deploy --env prod --canary 100 --push` : redéploie la bonne version à 100 % après le rollback.
- `cat reports/m7/canary-analysis.md` : relit la trace écrite de la décision.
- `git switch -c lab/m7-observabilite` puis `git add reports monitoring && git commit -m "docs(m7): …"` et `git push -u origin lab/m7-observabilite` : seconde branche avec les fichiers des étapes 3 à 10.
- `make check-m7` : vérification automatique finale (code retour = nombre de contrôles en échec).

Pièges / points d'attention

- **Ressources** : kube-prometheus-stack ≈ 2 Go + Loki/Alloy ≈ 0,5 Go ; prévoir ≥ 10 Go pour Docker. Libérer la mémoire des labs précédents (arrêter Airflow avec `(cd infra/airflow && docker compose down)`, mettre KFP en pause) plutôt que recréer le cluster.
- **Pods OOMKilled** : réduire la rétention Prometheus à 6 h, mettre KFP en pause, monter Docker à 12 Go.
- **node-exporter** `CreateContainerError : not a shared or slave mount` : sous Docker Desktop la racine du nœud kind n'est pas partagée ; garder `prometheus-node-exporter.hostRootFsMount.enabled: false` puis relancer `make monitoring`.
- `make monitoring` **en** `context deadline exceeded` : images longues à tirer au premier essai ; relancer (idempotent) et surveiller avec `kubectl get pods -n monitoring -w`.
- **PodMonitor non pris en compte** : `podMonitorSelectorNilUsesHelmValues` doit être `false`, sinon le label `release: kube-prom-stack` est exigé ; vérifier aussi le namespace (`scoring` et `scoring-staging`).
- **Cible** `scoring-api` **absente ou DOWN (404)** : service à zéro pod (staging) ou révision non instrumentée (image d'avant le Lab 7) — normal pendant un canary vers une ancienne image.
- `/metrics` **renvoie 307** : `app.mount("/metrics", make_asgi_app())` redirige vers `/metrics/` ; utiliser la route `GET /metrics` directe (ou `curl -L`).
- `Duplicated timeseries in CollectorRegistry` : métriques dans le registre global + `serve.py` importé deux fois (tests du Lab 6) ; utiliser un registre propre au module.
- **Compteurs incohérents avec plusieurs workers uvicorn** : chaque processus a ses propres compteurs ; un worker par pod (Knative ajoute des pods) ou mode multiprocess.
- **Grafana « No data »** : fenêtre `rate()` trop courte ou absence de trafic ; passer de `[1m]` à `[5m]` au démarrage et relancer `traffic.py`.
- **Dashboard absent de Grafana** : ConfigMap sans le label `grafana_dashboard: "1"` ou JSON invalide ; inspecter les logs du conteneur `grafana-sc-dashboard`.

- **Job de drift** `NoSuchKey` : `train.csv` non poussé sur le S3 ou empreinte DVC différente ; refaire `dvc push` puis recréer la ConfigMap `scoring-drift-reference`.
- **Job de drift « Moins de 500 demandes »** : trafic insuffisant dans la dernière heure ou Loki vide ; relancer le trafic et vérifier la requête LogQL.
- **Job de drift** `CreateContainerConfigError` : Secret `s3-credentials` ou ConfigMap absent du namespace `monitoring`.
- **Evidently** `KeyError` : API 0.4 (`as_dict()`, `drift_by_columns`) utilisée avec Evidently 0.7 ; utiliser `snapshot.dict()` et épingler `evidently==0.7.23`.
- **Pushgateway 404 / connection refused** : URL incorrecte ; le nom complet est `prometheus-pushgateway.monitoring:9091`.
- **Loki pods Pending** : volume persistant demandé sans classe de stockage adaptée ; garder `singleBinary.persistence.enabled: false`.
- **Loki** `max entries limit per query exceeded` : `limit` > 5 000 ; paginer comme `calc_drift.py`.
- **Alertmanager** `config reload failed` : erreur dans l'`AlertmanagerConfig` (Secret introuvable, matcher invalide) ; vérifier les logs de l'operator et le Secret `slack-webhook` (clé `url`).
- **Slack** `invalid_payload` : template de message incorrect ; simplifier et tester d'abord avec webhook.site.
- **Webhook CT : 401** : `CT_WEBHOOK_TOKEN` absent ou différent entre serveur et appel ; faire `set -a && source .env && set +a` dans les deux terminaux.
- `canary_analysis.py` **: WAIT en boucle** : pas assez de trafic sur le canary (10 %) ; relancer `traffic.py` plus longtemps ou monter le palier.
- `canary_analysis.py --apply` **: code 2** : l'outil GitOps refuse un arbre modifié dans `k8s/overlays/` ou une branche `lab/*` (les opérations partent de `main`).
- **Dataset personnalisé** : tout est lu dans `dataset.yaml` (variables, classe positive, attributs sensibles) ; un attribut « d'audit seulement » absent des features n'est pas envoyé à l'API et ne peut pas être suivi par groupe ; la classe positive n'est un « refus » que pour le crédit ; le seuil PSI 0,25 suppose une référence de quelques centaines de lignes.

Fiche générée à partir de Lab7-Enonce.pdf et Lab7-Commandes.pdf — cours MLOps.

Lab 8 — Audit de gouvernance du modèle de scoring · outils & commandes

Ce lab fait passer le modèle de scoring de crédit par une revue complète de gouvernance : sécurité de l'image et des modèles, mesure puis atténuation des disparités selon les attributs sensibles, explicabilité SHAP, Model Card et conformité AI Act / RGPD. La fiche « Commandes » du cours liste les commandes sans dire à quoi elles servent : cette fiche comble ce manque en expliquant l'utilité et le rôle de chaque outil, cible et commande.

Outils de la stack

Outil/service	Rôle / utilité
Trivy	Scanner de vulnérabilités (CVE) des images Docker ; compare l'image `scoring-api:1.0.0` du Lab 4 et l'image durcie, et sert de garde-fou en CI. Tourne en conteneur, version épinglée.
ModelScan	Détecte statiquement les opérateurs dangereux d'un fichier de modèle (pickle/joblib) ; son code retour est exploitable par la CI.
skops	Sérialisation sûre des modèles MLflow 3 ; `get_untrusted_types` liste les types que le fichier demande d'instancier, pour auditer le modèle du Registry.
Fairlearn	Mesure les disparités (métriques par groupe, parité démographique, règle des 4/5, equalized odds) et fournit les atténuations (`ThresholdOptimizer`, `ExponentiatedGradient`).
SHAP	Explique le modèle : importance globale (
Model Cards	Documentation standard du modèle en 11 sections, alimentée par les rapports produits.
External Secrets Operator (ESO)	Recopie des secrets d'un « coffre » vers Kubernetes : source de vérité unique, rotation et resynchronisation automatiques.
Docker / Docker Hardened Images (DHI)	Construction des images ; DHI fournit une base distroless (Python 3.12, `nonroot`, sans shell) pour durcir l'image de serving.
Kubernetes / kubectl / RBAC	Applique et vérifie les identités minimales (`ServiceAccount`, `Role`, `RoleBinding`) et les droits effectifs.
KServe (InferenceService)	Sert le modèle en production ; on lui assigne le ServiceAccount `scoring-sa`.
Argo CD	GitOps : applique l'état de `main` ; tout changement de prod passe par Git, pas par `kubectl patch`.
MLflow Registry	Source du champion (`scoring-model@champion`), de sa version et de son seuil de décision.

Outil/service	Rôle / utilité
SeaweedFS (S3)	Stockage objet des artefacts ; le secret `s3-credentials` du namespace `storage` sert de coffre simulé.
uv / pytest / git / gh	Environnement Python et scripts (`uv run`), tests K8s, versionnement et pull request.

Cibles Makefile et scripts

Cible / script	Rôle
`make check-m8`	Vérification automatique des livrables du lab ; le code retour est le nombre de contrôles en échec (0 = tout vert).
`make external-secrets`	Installe External Secrets Operator dans le cluster (étape 8).
`make pf`	Lance les port-forwards nécessaires à la séance.
`governance/__init__.py`	Paquet Python des scripts d'audit, lancés en module (`python -m governance.…`) depuis la racine, comme `src/`.
`governance.fairness_analysis`	Mesure les disparités pour chaque attribut sensible de `dataset.yaml` au seuil du champion ; écrit `reports/fairness_baseline.json`.
`governance.fairness_mitigation`	Compare trois atténuations (sans l'attribut, ThresholdOptimizer, ExponentiatedGradient) ; écrit `reports/fairness_mitigated.json`.
`governance.explain`	Importance globale SHAP ; produit `reports/shap_summary_bar.png`, `shap_beeswarm.png` et `shap_importance.json`.
`governance.explain_local`	Trois explications locales (waterfall) + motifs rédigés ; produit `reports/shap_waterfall_*.png` et `explanations_local.md`.
`packaging/scan-demo/make_pickles.py`	Génère deux pickles de démonstration (un sain, un piégé) pour ModelScan.
`packaging/sample_request.py`	Produit une requête d'exemple pour tester l'API `/predict`.
`packaging/Dockerfile.distroless`	Construit l'image de serving durcie (stage builder `-dev` + runtime distroless).
`tests/test_k8s.py`	Tests des overlays Kustomize (dont le `serviceAccountName`).

Commandes clés, expliquées

Branche de travail

- `cd ~/mlops-cours-2026` : racine du fork ; toutes les commandes s'y exécutent.
- `git switch main && git pull` : met `main` à jour avant de partir d'une base saine.
- `git switch -c lab/m8-governance` : crée la branche de travail du lab (convention Lab 0).

Étape 1.1 — Préparation

- `set -a && source .env && set +a` : charge et exporte les variables d'environnement pour la session.
- `make pf` : démarre les port-forwards (dont MLflow).
- `mkdir -p reports model-card governance k8s` : crée l'arborescence des livrables.
- `uv run python -c "import fairlearn, shap; print(fairlearn.__version__, shap.__version__)"` : vérifie la présence des bibliothèques d'audit (0.14.0 / 0.52.0).
- `trivy() { docker run ... aquasec/trivy:0.75.0 "$@" }` : encapsule Trivy dans un conteneur, version épinglée, avec le volume `trivy-cache` qui conserve la base de vulnérabilités entre deux scans.

Étape 1.2 — Scan Trivy de l'image du Lab 4

- `docker pull localhost:5001/scoring-api:1.0.0` : récupère l'image dans le démon local (Trivy le lit via le socket Docker).
- `trivy image localhost:5001/scoring-api:1.0.0 > reports/trivy_full.txt` : scan complet, toutes sévérités.
- `trivy image --severity HIGH,CRITICAL localhost:5001/scoring-api:1.0.0 > reports/trivy_critical.txt` : scan focalisé sur les vulnérabilités graves.
- `grep "^Total" reports/trivy_full.txt reports/trivy_critical.txt` : extrait les compteurs par cible scannée (système Debian, puis paquets Python).
- `grep -c "CVE-" reports/trivy_critical.txt` : compte les CVE listées.
- `trivy image --severity HIGH,CRITICAL --ignore-unfixed ...` : ne montre que les CVE corrigeables aujourd'hui.
- `docker buildx imagetools inspect aquasec/trivy:0.75.0` : donne le `sha256:…` pour épingler l'image par empreinte en CI.

Étape 1.3 — Corriger via une base distroless

- `docker login dhi.io` : s'authentifie au registre DHI (compte Docker gratuit + jeton d'accès personnel).
- `MODEL_VERSION=$(uv run python -c "import json; print(json.load(open('packaging/model/metadata.json'))['model_version'])")` : lit la version du modèle pour l'injecter au build.
- `docker build -f packaging/Dockerfile.distroless -t scoring-api:distroless --build-arg VERSION=1.0.0 --build-arg MODEL_VERSION="$MODEL_VERSION" .` : construit l'image durcie depuis la racine du repo.
- `docker images scoring-api` : liste les images (taille, tag).
- `docker run --rm --entrypoint python scoring-api:distroless --version` : vérifie le Python embarqué (3.12.x).
- `docker inspect -f '{{.Config.User}}' scoring-api:distroless` : vérifie l'utilisateur non-root (65532:65532).
- `docker run --rm --entrypoint sh scoring-api:distroless -c "echo shell" || echo "pas de shell dans l'image"` : confirme l'absence de shell (échec attendu).
- `uv run python packaging/sample_request.py > /tmp/request.json` : prépare une requête de test.
- `docker run -d --rm -p 8001:8000 --name scoring-distroless scoring-api:distroless` : démarre l'API durcie.

- `curl -s http://localhost:8001/health` : vérifie la santé de l'API.
- `curl -s -X POST http://localhost:8001/predict -H "Content-Type: application/json" -d @/tmp/request.json` : compare la prédiction à celle du Lab 4.
- `docker inspect -f '{{.State.Health.Status}}' scoring-distroless` : lit l'état du HEALTHCHECK.
- `docker stop scoring-distroless` : arrête le conteneur.
- `trivy image --severity HIGH,CRITICAL scoring-api:distroless > reports/trivy_distroless.txt` : re-scan de l'image durcie.
- `echo "AVANT : $(grep -c CVE- reports/trivy_critical.txt)"` / `echo "APRÈS : ..."` : compare le nombre de CVE avant/après.
- `docker tag scoring-api:distroless localhost:5001/scoring-api:1.0.0-distroless` puis `docker push ...` : publie l'image durcie à côté de la 1.0.0.
- `docker images --digests localhost:5001/scoring-api` : récupère le digest qui identifie réellement l'image auditée.

Étape 2 — ModelScan : pickle vs ONNX

- `uv run python packaging/scan-demo/make_pickles.py` : génère les deux pickles de démonstration (sain / piégé).
- `uv run modelscan -p packaging/scan-demo/ > reports/modelscan_pickle.txt` : scanne le pickle piégé (détecte un CRITICAL `posix.system`).
- `uv run modelscan -p packaging/model/model.onnx > reports/modelscan_onnx.txt` : scanne l'ONNX (« No issues found », en fait « skipped »).
- `grep -E "Total Issues|CRITICAL|Description" reports/modelscan_pickle.txt` : lit le résultat exploitable.
- `for f in packaging/scan-demo/model.pkl packaging/scan-demo/malicious.pkl packaging/model/model.onnx; do uv run modelscan -p "$f" > /dev/null 2>&1; echo "$f -> code $?"; done` : montre les codes retour (0, 1, 3) — la CI doit échouer si code 1 ou 2.
- `uv run modelscan -p packaging/scan-demo/ -r json -o reports/modelscan_pickle.json` : rapport JSON exploitable par un outil.
- `uv run mlflow artifacts download -u models:/scoring-model@champion -d /tmp/champion` : télécharge le champion du Registry.
- `uv run python -c "from skops.io import get_untrusted_types; print(get_untrusted_types(file='/tmp/champion/model.skops'))"` : liste les types demandés par le fichier skops (attendu `['numpy.dtype']`).

Étape 3 — Fairlearn : mesurer les disparités

- `grep -A6 "^sensitive:" dataset.yaml` : affiche la section des attributs sensibles (le script ne code aucun nom en dur).
- `uv run python -m governance.fairness_analysis | tee reports/fairness_baseline.txt` : lance l'analyse, affiche le rapport et l'archive.

Étape 4 — Atténuation des disparités

- `uv run python -m governance.fairness_mitigation | tee reports/fairness_mitigated.txt` : compare les stratégies et chiffre le compromis coût métier / équité.

Étape 5 — SHAP

- `uv run python -m governance.explain` : importance globale + graphiques (bar, beeswarm, JSON).
- `uv run python -m governance.explain_local` : trois explications locales (refus net, acceptation nette, cas limite) + motifs rédigés.
- `uv run python -m governance.explain_local --id 819` : explique une demande précise du jeu de test.
- `ls reports/shap_waterfall_*.png` : vérifie les fichiers produits.

Étape 6 — Générer la Model Card

- `cp labs/m8-governance/starter/model-card.md model-card/model-card.md` : part du squelette (11 sections).
- `grep "^## " model-card/model-card.md` : liste les sections du document.
- heredoc `uv run python - <<'EOF' ... EOF` : génère les tableaux par groupe depuis `reports/fairness_baseline.json`, à coller dans la section 4 (chiffres réels, jamais recopiés).

Étape 7 — RBAC Kubernetes minimal

- `kubectl apply -f k8s/rbac-scoring.yaml` : crée le `ServiceAccount` `scoring-sa`, le `Role` `scoring-reader` et le `RoleBinding`.
- `kubectl get serviceaccount scoring-sa -n scoring` : vérifie la création.
- `kubectl kustomize k8s/overlays/prod | grep serviceAccountName` : rend l'overlay de prod et vérifie le patch.
- `uv run pytest -q tests/test_k8s.py` : exécute les tests des overlays.
- `git add k8s/rbac-scoring.yaml k8s/overlays/prod/kustomization.yaml` puis `git commit -m "feat(k8s): ..."` : versionne le RBAC et le patch prod.
- `SA=system:serviceaccount:scoring:scoring-sa` : identité sous laquelle on teste les droits.
- `kubectl auth can-i create pods --as=$SA -n scoring` (attendu no), `list secrets` (no), `get secret/s3-credentials -n scoring` (yes), `get secret/s3-credentials -n storage` (no) : vérifie le moindre privilège.

Étape 8 — ExternalSecret pour les identifiants S3

- `make external-secrets` : installe External Secrets Operator.
- `kubectl get pods -n external-secrets` : vérifie les trois déploiements (contrôleur, webhook, cert-controller).
- `kubectl apply -f k8s/mlflow-creds-source.yaml` : crée ServiceAccount, Role, RoleBinding, `SecretStore` et `ExternalSecret`.
- `kubectl wait --for=condition=Ready secretstore/course-secrets -n scoring --timeout=60s` : attend que le SecretStore soit prêt.
- `kubectl wait --for=condition=Ready externalsecret/s3-credentials -n scoring --timeout=60s` : attend la synchronisation du Secret.
- `kubectl get secretstore,externalsecret -n scoring` : état des objets.
- `kubectl get secret s3-credentials -n scoring -o jsonpath='{.metadata.ownerReferences[0].kind}{"\n"}{.metadata.annotations}{"\n"}'` : vérifie que le Secret appartient à l'ExternalSecret et porte les annotations KServe.

- `for ns in storage scoring; do kubectl get secret s3-credentials -n $ns -o jsonpath='{.data.AWS_ACCESS_KEY_ID}' | base64 -d; echo " ($ns)"; done` : compare les valeurs à la source de vérité.
- `kubectl delete secret s3-credentials -n scoring` puis `sleep 10 && kubectl get secret s3-credentials -n scoring` : montre que l'opérateur recrée le Secret.
- `kubectl get externalsecret s3-credentials -n scoring` : reste en `SecretSynced`.

Étape 9 — Conformité AI Act, PR et vérification

- `cp labs/m8-governance/starter/compliance-ai-act.md reports/compliance-ai-act.md` : part du squelette (5 sections + sources).
- `git add governance model-card k8s packaging/Dockerfile.distroless packaging/Dockerfile.distroless.dockerignore reports ':(exclude)packaging/scan-demo/*.pkl'` : committe l'audit sans les pickles de démonstration.
- `git commit -m "feat(lab8): audit de gouvernance (...)"` / `git push -u origin lab/m8-governance` / `gh pr create` : pousse la branche et ouvre la pull request.
- `kubectl annotate application scoring-prod -n argocd argocd.argoproj.io/refresh=normal --overwrite` : force Argo CD à resynchroniser sans attendre.
- `kubectl wait isvc/scoring-model -n scoring --timeout=180s --for=jsonpath='{.spec.predictor.serviceAccountName}'=scoring-sa` : attend le nouveau ServiceAccount sur l'InferenceService.
- `kubectl wait --for=condition=Ready isvc/scoring-model -n scoring --timeout=180s` : attend que l'InferenceService soit prêt.
- `POD=$(kubectl get pods -n scoring -l serving.kserve.io/inferenceservice=scoring-model --sort-by=.metadata.creationTimestamp -o name | tail -1)` : récupère le pod de la nouvelle révision.
- `kubectl get "$POD" -n scoring -o jsonpath='{.spec.serviceAccountName}{"\n"}'` : confirme que le pod tourne sous `scoring-sa`.
- `kubectl get "$POD" -n scoring -o yaml | grep -c kube-api-access` : résultat 0 = aucun jeton monté.
- `make check-m8` : vérification automatique finale (code retour = nombre d'échecs).

Pièges / points d'attention

- **Trivy :** `TOOMANYREQUESTS` au téléchargement de la base — limite de débit du registre. Relancer plus tard ; la base est en cache dans `trivy-cache`, `--skip-db-update` réutilise le cache local.
- **Trivy :** `unable to find the specified image` — l'image n'est pas dans le démon Docker (Trivy le lit via le socket) : faire `docker pull localhost:5001/scoring-api:1.0.0` avant de scanner.
- `docker build : unauthorized / pull access denied` **sur** `dhi.io` — pas connecté au registre DHI : `docker login dhi.io` avec identifiant et jeton personnel.
- **Distroless :** `ModuleNotFoundError` **au démarrage** — venv non copié ou builder/runtime sur des Python différents : vérifier `COPY --from=builder /opt/venv /opt/venv`, `ENV PATH` et les deux `FROM` en `3.12-debian13`.
- `docker exec … sh : executable file not found` — normal, l'image distroless n'a pas de shell : déboguer avec `--entrypoint python ... -c "…"` ou une image `-dev`.

- **ModelScan : « No issues found » mais rien d'analysé** — format non supporté (ONNX, skops) : fichier « skipped », code retour 3 ; utiliser `--show-skipped`, et `skops.io.get_untrusted_types` pour skops.
- **ModelScan :** `argument of type 'NoneType' is not iterable` — pickle écrit par joblib (bug de ModelScan 0.8.8) : produire le pickle avec `pickle.dump`.
- `MlflowException: Registered model alias champion not found` — Lab 1 non terminé ou port-forward arrêté : `make pf`, puis vérifier l'alias dans l'UI MLflow.
- `KeyError: 'decision_threshold'` — champion entraîné avant l'ajout du paramètre : `dvc repro` puis `python -m src.promote`.
- `ModuleNotFoundError: No module named 'governance'` — script lancé par son chemin ou hors racine : lancer depuis la racine `uv run python -m governance.fairness_analysis`.
- **Fairlearn :** `ValueError` **sur** `sensitive_features` — attribut absent du test ou `bins` ne couvrant pas toutes les valeurs (NaN) : vérifier `sensitive` dans `dataset.yaml` ; les bins doivent englober min et max.
- **Un groupe affiche des taux à 0 % ou 100 %** — groupe minuscule : lire l'IC 95 % et l'alerte « moins de 30 demandes » ; non concluant, à documenter.
- **ThresholdOptimizer :** `not binary` **ou forme inattendue** — `predict_proba` ne renvoie pas `(n, 2)` : le modèle doit être un classifieur binaire (cible encodée 0/1).
- **SHAP :** `Additivity check failed in TreeExplainer` — explainer « interventionnel » sur des arbres scikit-learn : utiliser `shap.TreeExplainer(modèle)` sans données de fond.
- **SHAP waterfall :** `IndexError` **ou valeurs 3D** — classifieur à deux sorties `(n, colonnes, 2)` : prendre la classe positive (`sv[..., 1]`).
- **RoleBinding refusé ou sans effet** — mauvais namespace : le RoleBinding doit être dans le même namespace que le Role ; le sujet peut venir d'un autre namespace.
- `kubectl patch isvc` **sans effet durable** — Argo CD ré-applique le manifest de Git : modifier `k8s/overlays/prod/kustomization.yaml` et fusionner la PR.
- **ISVC toujours sur** `default` **après la fusion** — PR non fusionnée, Argo CD non synchronisé, ou patch mal indenté : `kubectl kustomize k8s/overlays/prod | grep serviceAccountName`, vérifier l'Application `Synced`, forcer l'annotation refresh.
- **ExternalSecret :** `SecretSyncedError` — SecretStore invalide ou droits de lecture manquants : `kubectl describe externalsecret` / `secretstore`, vérifier le Role `eso-s3-reader` du namespace `storage`.
- **SecretStore :** `could not verify if client is valid` — ESO n'obtient pas de jeton pour `eso-s3-reader` ou CA introuvable : vérifier le ServiceAccount dans `scoring` et la ConfigMap `kube-root-ca.crt`.
- `no matches for kind "ExternalSecret" in version "external-secrets.io/v1beta1"` — ancien manifest : `v1beta1` n'est plus servie, passer en `external-secrets.io/v1`.
- `auth can-i : yes` **sur** `create pods` — test lancé avec votre identité ou droits hérités : tester avec `--as=system:serviceaccount:scoring:scoring-sa`, puis `kubectl get clusterrolebinding -o wide | grep scoring-sa`.
- **Un tag n'est pas une preuve** — `scoring-api:distroless` peut être reconstruite demain avec un autre contenu : ce qui identifie l'image auditée est son digest ; le scanner lui-même peut être compromis (Trivy piégé en 2026), d'où l'épinglage par version postérieure à l'incident et, en CI, par digest.
- **Ne jamais charger** `malicious.pkl` — un pickle exécute du code arbitraire à son chargement ; ModelScan détecte des motifs statiques mais ne garantit pas l'absence de tout risque.

- **Pour Fairlearn, « sélectionner » = REFUSER** — la classe positive est le risque (`bad`) ; la règle des 4/5 se calcule sur les taux d'acceptation (l'issue favorable).
- **Auditer au bon seuil** — le modèle ne décide pas à 0,5 mais à `decision_threshold` (0,318) : auditer à ce seuil, sinon on audite un autre modèle.
- **Retirer l'attribut sensible ne suffit pas** (« fairness through unawareness ») — les proxies (AUC 0,71-0,78) permettent de deviner le groupe ; « ThresholdOptimizer » avec son objectif par défaut (accuracy) double le coût métier et applique un seuil différencié par groupe (juridiquement risqué).
- **Sécurité des jetons** — par défaut, les pods reçoivent un jeton d'API monté, inutile et donc dangereux : la défense en profondeur vise `automountServiceAccountToken: false` et `can-i : no/no/yes`.
- **SHAP décrit le modèle, pas une causalité** — l'explication due au demandeur (RGPD art. 22) doit être compréhensible, pas une formule.
- **Model Card Toolkit non maintenu** — générer les tableaux dans la CI, le texte d'analyse reste écrit par l'équipe.
- **Dataset personnel** — déclarer tous les attributs protégés dans `sensitive`, garder `target.positive_class` = classe défavorable, regrouper les modalités rares (< 30 individus = taux inexploitables).

Fiche générée à partir de Lab8-Enonce.pdf et Lab8-Commandes.pdf — cours MLOps.

Lab 9 — RAG MLOps évalué et observé · outils & commandes

Ce lab guide construit un système RAG sur la documentation du cours MLOps : on sert un LLM local (Ollama sur CPU ou vLLM sur GPU), on indexe les documents dans une base vectorielle Qdrant, on assemble une chaîne RAG LangChain avec citations de sources, puis on l'évalue automatiquement (Ragas), on l'observe (tracing Phoenix, métriques Prometheus) et on teste sa résistance au prompt injection. Contrairement aux labs du socle, tout tourne sur le poste, dans le dossier `rag/`, sans cluster kind.

Outils de la stack

Outil/service	Rôle / utilité
Ollama	Sert un LLM local, sur CPU ou GPU grand public (llama.cpp, modèles quantifiés 4 bits). Option A, recommandée pour le lab : un utilisateur, `qwen3.5:4b` (~20-40 s par requête).
vLLM	Sert un LLM sur GPU pour beaucoup d'utilisateurs en parallèle (PagedAttention, continuous batching, tensor parallelism) ; standard de la prod. Option B, GPU NVIDIA 16 Go minimum (~1-3 s par requête).
Ollama / `vLLM` (API)	Exposent tous deux une API compatible OpenAI (`/v1/chat/completions`, `/v1/models`) : le code du lab fonctionne avec l'un ou l'autre.
Qdrant	Base vectorielle en Docker qui stocke les embeddings des documents du cours ; sert l'index et la recherche de proximité.
LangChain (`langchain-core`, LCEL)	Framework d'assemblage de la chaîne RAG : prompts, retrieval, passage à travers des étapes traçables (`RunnablePassthrough.assign`).
langchain-community / langchain-text-splitters	Fournit `PyPDFLoader`, `TextLoader`, `FastEmbedEmbeddings` et `RecursiveCharacterTextSplitter` (découpage en chunks).
langchain-qdrant / qdrant-client	Connecteur entre LangChain et Qdrant (upsert des chunks, retriever).
langchain-ollama / langchain-openai	Adaptateurs du LLM pour Ollama ou vLLM côté LangChain.
fastembed	Calcule les embeddings en ONNX sur CPU sans PyTorch (`bge-small-en-v1.5`, 384 dim., ~70 Mo).
pypdf	Lecture des fichiers PDF.
Ragas	Évaluation du RAG par LLM-juge : `faithfulness`, `answer_relevancy`, `context_precision`, `context_recall`.
Phoenix (Arize)	Serveur de traces OpenTelemetry spécialisé LLM : visualise chaque requête (retrieval, prompt, réponse, tokens, latence).
arize-phoenix-otel / openinference-instrumentation-langchain	Client OpenTelemetry et auto-instrumentation de LangChain pour envoyer les traces à Phoenix.
LangSmith	Alternative managée (SaaS) de tracing : mêmes chaînes tracées sans opérer de serveur, au prix de l'envoi des prompts/documents à un tiers.

Outil/service	Rôle / utilité
FastAPI + uvicorn	Exposent la chaîne RAG en API (`/ask`, `/health`) ; `uvicorn` est le serveur ASGI.
prometheus_client	Expose des métriques métier scrapables par Prometheus (`rag_requests`, `rag_request_duration_seconds`, `rag_llm_tokens`), pour rejoindre le monitoring du Module 7.
Docker	Fait tourner Qdrant (et Ollama en variante conteneurisée).

Cibles Makefile et scripts

Cible / script	Rôle
`make check-m0` … `make check-m8`	Vérifications du socle des labs précédents. Le Lab 9 ne fait pas partie de ce socle.
`make check-m9`	Vérifie le Lab 9 de bout en bout, sans rien modifier : LLM, Qdrant, API RAG, golden set, résultats Ragas, traces Phoenix, logs d'injection et rapport.
`labs/m9-llmops/check.sh`	Script appelé par `make check-m9` ; affiche ✔/✘ par item, le code retour est le nombre d'échecs (attendu : 17/17).
`rag/config.py`	Configuration commune : Qdrant, embeddings et LLM en un seul endroit, piloté par variables d'environnement.
`rag/ingest.py`	Indexe les documents (PDF/MD, ou doc du projet) dans Qdrant, en chunks.
`rag/rag.py`	Chaîne RAG LCEL (retrieval top-5 + prompt + LLM), avec sources tracées.
`rag/tracing.py`	Active le tracing Phoenix et auto-instrumente LangChain (importé quand `PHOENIX_TRACING=1`).
`rag/evaluate_rag.py`	Génère les réponses sur le golden set et calcule les 4 métriques Ragas ; peut servir de gate en CI.
`rag/phoenix_stats.py`	Calcule depuis les traces Phoenix les tokens, coût équivalent et latences P50/P95.
`rag/api.py`	Expose le RAG en FastAPI avec métriques Prometheus, en passant par le guardrail.
`rag/guardrails.py`	Guardrail d'entrée (patterns d'injection) et de sortie (fuite de secret/prompt), via `safe_query`.
`rag/injection_test.py`	Joue une liste d'attaques de prompt injection contre la chaîne (avant guardrail).
`rag/injection_test_guarded.py`	Rejoue les attaques plus une requête légitime, via `safe_query` (après guardrail).
`rag/golden_set.json`	Jeu de référence de Q/R (26 questions fournies, ≥ 20 exigées) pour évaluer le RAG.
`rag/RAPPORT.md`	Trame de rapport à compléter (9 sections).

Cible / script	Rôle
`rag/reports/`	Sorties : `ragas_results.csv`, `ragas_summary.json`, `ragas_run.log`, `phoenix_stats.json`, `injection_before.log`, `injection_after.log`, `phoenix_trace.png`.
`rag/.gitignore`	Exclut `docs/*.pdf` : les supports du cours ne sont pas republiés dans le fork public.

Commandes clés, expliquées

Préparation (branche de travail)

- `git switch main && git pull` puis `git switch -c lab/m9-llmops` : on part d'un `main` à jour et on crée la branche dédiée au lab.
- `kubectl get nodes` et `make pf` : vérifications de début de séance (le cluster n'est pas utilisé par le lab).

Étape 1 — Installer et servir le LLM

- `curl -fsSL https://ollama.com/install.sh | OLLAMA_VERSION=0.35.1 sh` : installe Ollama en épinglant la version du cours (Linux/WSL2).
- `ollama --version` : confirme la version installée (attendu `0.35.1`).
- `ollama pull qwen3.5:4b` : télécharge le modèle (~3,4 Go).
- `ollama serve &` (ou `sudo systemctl start ollama`) : démarre le serveur sur `localhost:11434`.
- `ollama run qwen3.5:4b --think=false "…"` : teste le modèle sans phase de « réflexion » (réponse directe).
- `docker run -d --name ollama -p 11434:11434 -v ollama:/root/.ollama ollama/ollama:0.35.1` puis `docker exec ollama ollama pull qwen3.5:4b` : variante Ollama en conteneur.
- `uv venv ~/vllm-env --python 3.12` puis `source ~/vllm-env/bin/activate` et `uv pip install vllm==0.31.0` : environnement GPU séparé pour vLLM (PyTorch/CUDA).
- `vllm serve Qwen/Qwen3.5-4B --host 0.0.0.0 --port 8000 --max-model-len 16384 --gpu-memory-utilization 0.9 --language-model-only --reasoning-parser qwen3 --default-chat-template-kwargs '{"enable_thinking": false}'` : lance le serveur compatible OpenAI sans thinking.
- `curl http://localhost:8000/v1/models` : vérifie que vLLM répond.

Étape 2 — Lancer Qdrant

- `docker run -d --name qdrant -p 6333:6333 -p 6334:6334 -v ~/qdrant-storage:/qdrant/storage qdrant/qdrant:v1.19.2` : lance Qdrant (REST 6333, gRPC 6334) avec stockage persistant.
- `curl http://localhost:6333/` : vérifie la version JSON renvoyée ; UI web sur `http://localhost:6333/dashboard`.

Étape 3 — Ingérer les documents

- `mkdir -p rag/docs` puis `cp ~/Downloads/MLOps_*.pdf rag/docs/` et `ls -lh rag/docs/` : prépare et vérifie le corpus.
- `cd rag` puis `uv sync` : crée l'environnement isolé du RAG depuis `rag/pyproject.toml`.

- `uv run python -c "import ragas, phoenix; print(ragas.__version__, phoenix.__version__)"` : vérifie les dépendances (attendu `0.4.3 20.19.0`).
- `uv run python ingest.py` (ou `uv run python ingest.py --with-labs`) : indexe les documents dans Qdrant ; `--with-labs` indexe les énoncés `labs/*/README.md` sans les PDF.
- `QDRANT_COLLECTION=project_docs uv run python ingest.py --project` : indexe la doc de votre propre projet dans une collection séparée.
- `QDRANT_COLLECTION=project_docs uv run python rag.py` : interroge cette collection projet (après l'étape 4).

Étape 4 — Construire la chaîne RAG

- `uv run python rag.py` : exécute la chaîne LCEL sur une question de test, affiche la réponse et 3 sources.

Étape 5 — Créer un golden set

- `cp ../labs/m9-llmops/starter/golden_set.json .` : copie le golden set fourni.
- `uv run python -c "import json; print(len(json.load(open('golden_set.json'))), 'questions')"` : compte les questions.

Étape 6 — Évaluer avec Ragas

- `mkdir -p reports` : prépare le dossier des sorties.
- `RAGAS_LIMIT=5 uv run python evaluate_rag.py 2>&1 | tee reports/ragas_run.log` : premier passage rapide sur 5 questions, journalisé.
- `RAGAS_MIN_FAITHFULNESS=0.7 uv run python evaluate_rag.py` : renvoie un code retour 1 si la `faithfulness` moyenne passe sous le seuil (gate CI).

Étape 7 — Tracing Phoenix + API

- `uv run phoenix serve` : lance le serveur de traces (`http://localhost:6006`, OTLP sur `/v1/traces`).
- `PHOENIX_TRACING=1 uv run python rag.py` : relance la chaîne avec le tracing activé.
- `uv run python phoenix_stats.py` : calcule tokens, coût équivalent et P50/P95 depuis les traces et écrit `reports/phoenix_stats.json`.
- `PHOENIX_TRACING=1 uv run uvicorn api:app --port 8090 &` : démarre l'API RAG avec métriques.
- `curl -s -X POST localhost:8090/ask -H 'Content-Type: application/json' -d '{"question": "Que fait PagedAttention dans vLLM ?"}'` : interroge l'API.
- `curl -s localhost:8090/metrics | grep '^rag_'` : lit les métriques Prometheus exposées.
- `export LANGSMITH_TRACING=true` / `export LANGSMITH_API_KEY=lsv2_...` / `export LANGSMITH_PROJECT=rag-mlops` puis `uv run python rag.py` : variante de tracing managée LangSmith.

Étape 8 — Prompt injection + guardrail

- `uv run python injection_test.py 2>&1 | tee reports/injection_before.log` : démontre l'attaque (4 prompts).
- `uv run python injection_test_guarded.py 2>&1 | tee reports/injection_after.log` : rejoue via le guardrail (3 bloquées, requête légitime passée).

Étape 9 — Rapport final et versionnage

- `cp ../labs/m9-llmops/starter/RAPPORT.md .` : copie la trame du rapport.
- `cd ..`, `git add rag/`, `git status --short` (aucun PDF), `git commit -m "Lab 9 : RAG évalué (Ragas) et observé (Phoenix)"`, `git push -u origin lab/m9-llmops`, `gh pr create` : commit et PR.
- `docker start qdrant` puis `cd rag && uv sync`, `uv run python ingest.py`, `PHOENIX_TRACING=1 uv run python rag.py` : remise à niveau d'un second poste.
- `make check-m9` : vérification automatique (attendu `★ 17/17 vérifications OK`).

Pièges / points d'attention

- Ollama `model not found` : le modèle n'est pas téléchargé ou `LLM_MODEL` diffère ; faire `ollama pull qwen3.5:4b` puis `ollama list` et vérifier `echo $LLM_MODEL`.
- Réponses Ollama très lentes : CPU faible ou modèle qui « réfléchit » ; utiliser `qwen3.5:0.8b` et vérifier que `reasoning=False` est bien passé.
- Contexte tronqué / réponses incohérentes : fenêtre par défaut de 4K tokens sur CPU ; relancer avec `OLLAMA_CONTEXT_LENGTH=8192 ollama serve`.
- Qdrant `connection refused` ou `collection not found` : conteneur non démarré (`docker start qdrant`) ou nom de collection différent entre ingestion et requête (garder le même `QDRANT_COLLECTION` partout et relancer `ingest.py`).
- Embeddings : échec de téléchargement (pas d'accès à Hugging Face, proxy) → définir `HF_ENDPOINT` vers un miroir ; embeddings lents → rester sur `bge-small-en-v1.5`, pas `bge-large`, sur CPU.
- RAG sources vides : collection vide ou `k=0` ; vérifier le nombre de points dans l'UI Qdrant et `search_kwargs={"k": 5}`.
- Réponse hors sujet ou obsolète : prompt trop permissif ou ancien document indexé ; un RAG n'est jamais plus à jour que son index.
- Imports LangChain : `ModuleNotFoundError: langchain.chains` = code écrit pour 0.x ; utiliser les imports 1.x ou `langchain_classic` ; `No module named 'langchain_community.chat_models.vertexai'` = `langchain-community` 0.4.2 installé à la main → `cd rag && uv sync` (épingle 0.4.1).
- Ragas lent (9 appels au juge par question) : utiliser `RAGAS_LIMIT=5` ou un juge plus rapide ; un échec `ValidationError` isolé (JSON invalide) donne un `NaN`, normal de temps en temps.
- Phoenix : port 6006 occupé → `PHOENIX_PORT=6007 uv run phoenix serve` et adapter `PHOENIX_COLLECTOR_ENDPOINT` ; aucune trace → `PHOENIX_TRACING=1` et Phoenix lancé avant le script.
- `make check-m9` 5a/5b en échec : l'API RAG n'est pas lancée (`cd rag && PHOENIX_TRACING=1 uv run uvicorn api:app --port 8090 &`).
- Avertissement `VIRTUAL_ENV … does not match` : le venv du socle est activé, `uv` utilise `rag/.venv` — sans conséquence (`deactivate` fait taire le message).
- Ne jamais indexer de secrets : tout ce qui est indexé peut sortir (LLM02). Le guardrail à base de regex se contourne et produit des faux positifs ; en prod, combiner guardrail d'entrée + de sortie + monitoring.

Fiche générée à partir de Lab9-Enonce.pdf et Lab9-Commandes.pdf — cours MLOps.

Lab 10 — Mini-projet MLOps · outils & commandes

Le mini-projet consiste à faire passer un modèle du notebook à la production, à deux, de façon reproductible, observable, auditable et éthique. Le repo construit lab après lab (Labs 0 à 8) forme le socle : il est déployé chez un fournisseur cloud, complété par une extension individuelle, puis défendu en soutenance avec un défi en direct. Cette fiche explique l'utilité et le rôle de chaque outil, cible et commande regroupés dans la fiche « Commandes » du cours.

Outils de la stack

Outil/service	Rôle / utilité
`kind`	Cluster Kubernetes local : fait tourner tout le socle, c'est là que passent les checks et le défi en direct.
`kubectl`	Pilotage du cluster : observer pods, événements, logs, décrire une ressource, lire l'état d'un service.
`git` / GitHub	Versionner le code, gérer branches et PR, porter les tags de jalon, héberger le repo et les règles de protection.
`dvc`	Versionner les données et le modèle, rejouer le pipeline (`repro`), pousser/tirer vers le remote DVC.
MLflow	Tracer les runs et le registre de modèles ; gère l'alias `champion` et les versions.
KFP (Airflow)	Orchester le pipeline `scoring-pipeline` (Airflow utilisé comme point de comparaison).
KServe	Servir le modèle via une `InferenceService` (`scoring-model`, namespace `scoring`), autoscaling, canary.
ONNX Runtime	Exporter/exécuter le modèle de l'API (`scoring-api`), optimisation et quantification.
GitHub Actions	CI : build et test de l'image, publication sur GHCR, exécution des gates.
Argo CD	GitOps : synchronise le cluster depuis `main`, détecte les écarts (sync/diff).
Prometheus / Grafana	Collecte de métriques, tableaux de bord, alertes (avec runbook).
Evidently	Détection de drift des features et des prédictions.
GX	Validation des données (colonnes, domaines, gates du Lab 3).
Fairlearn	Analyse d'équité par groupe, base du gate d'équité.
SHAP	Explicabilité des décisions.
Trivy / ModelScan	Scans de sécurité de l'image et de l'artefact modèle.

Outil/service	Rôle / utilité
SeaweedFS (S3)	Stockage objet du cluster, sert de remote DVC (`s3://dvc-store/scoring` sur `localhost:8333`).
registry local	Stockage des images (`localhost:5001`) à côté de GHCR.
Fournisseur cloud (Azure, Google Cloud, AWS…)	Héberge l'API de scoring en ligne, en HTTPS, publiquement accessible.
`uv` / `uvx`	Gestion de l'environnement Python (`uv sync`, `uv run`) et exécution d'outils ponctuels sans install (`uvx`).
`jq`	Lecture et filtrage de réponses JSON (ex. `/metadata`).
`qrcode[pil]`	Génération des QR codes PNG du poster.
`gh` CLI	Vérification des relectures de PR `ext/*`.

Cibles Makefile et scripts

Cible / script	Rôle
`make up`	Démarre le cluster et l'infrastructure du socle.
`make pf`	Port-forward : rend accessibles les services du cluster (MLflow, etc.). À vérifier en début de séance.
`make down`	Arrête/retire l'environnement ; à manier avec précaution (à ne jamais utiliser pour « réparer »).
`make check-m0` … `make check-m8`	Vérification automatique d'une brique du socle (cluster, DVC/MLflow, pipeline, gates, image, service, CI/CD, monitoring, gouvernance).
`make check-mX`	Vérification générique d'une brique du socle.
`make check-m10`	Pré-contrôle de rendu du mini-projet ; par défaut le jalon « final », paramétrable par `JALON=`.
`./labs/m10-capstone/check.sh`	Pré-contrôle complet du rendu ; options `--sans-cluster` (repo seul) et `--gh` (relectures de PR).
`packaging/sample_request.py`	Produit une requête d'exemple pour tester l'API.
`src/dataset_config.py`	Lit `dataset.yaml` : tout nom de colonne, cible, classe ou seuil vient de là, rien en dur.
`src/download.py`	Télécharge et prépare le jeu de données (script de reproductibilité).
`docs/journal.md`	Journal de bord, une entrée datée par séance.
`docs/adr/`	Architecture Decision Records (3 minimum au rendu).
`docs/runbook.md`	Procédures d'exploitation par incident.

Commandes clés, expliquées

Mise en place et premier tour de piste

- `git clone git@github.com:PROPRIETAIRE/mlops-cours-2026.git` — clone le repo (le repo commun en option A, le sien en option B).
- `git remote add upstream https://github.com/mlops-itbs-2026/mlops-cours-2026.git` — ajoute le repo du cours comme remote de référence (corrections, `solution/*`).
- `cp .env.example .env` — crée la configuration locale propre à chaque membre.
- `make up && make pf` — démarre l'infrastructure puis ouvre les accès ; chacun a son propre cluster.
- `git switch -c lab/m0-journal` — crée une branche de travail pour le lab (convention `lab/mX-…`).
- `$EDITOR docs/journal.md` — rédige l'entrée du journal de bord.
- `git add docs/journal.md` / `git commit -m "lab/m0 : journal de bord"` — indexe et commite la modification.
- `git push -u origin lab/m0-journal` — pousse la branche et déclenche la proposition de PR (relue par le partenaire par convention).

Rendre un jalon

- `git switch main && git pull` — se place sur un `main` à jour avant le jalon.
- `make check-m10 JALON=j1` — lance le pré-contrôle correspondant au jalon visé (ici J1).
- `git tag -a j1 -m "J1 MVP : pipeline avec gate, modèle servi par KServe"` — pose un tag annoté sur le commit qui fait foi.
- `git push origin j1` — pousse le tag ; un tag poussé ne se déplace pas, il faut corriger via un nouveau commit sur `main`.

Vérifier le déploiement cloud

- `URL=https://votre-api.exemple` — définit l'URL donnée par le fournisseur.
- `curl -fsS "$URL/health"` — vérifie que l'API répond (`{"status":"ok","model_loaded":true}`).
- `curl -fsS "$URL/metadata" | jq .model` — lit la version servie, à comparer à `scoring-model@champion`.
- `uv run python packaging/sample_request.py > /tmp/request.json` — génère une requête d'exemple.
- `curl -fsS -X POST "$URL/predict" -H "Content-Type: application/json" -d @/tmp/request.json` — envoie une prédiction à l'API.
- Ouvrir `$URL/docs` depuis un téléphone, hors Wi-Fi de l'école — vérifie l'essai d'une prédiction et la page de documentation.

Diagnostiquer (méthode en cinq temps)

- `kubectl get pods -A | grep -v Running` — repère en un coup d'œil tout ce qui ne tourne pas, dans tous les namespaces.
- `kubectl -n scoring get events --sort-by=.lastTimestamp` — reconstitue le récit chronologique de ce qui s'est passé.
- `kubectl -n scoring describe pod <pod>` — inspecte conditions, sondes et montages du pod.
- `kubectl -n scoring logs <pod> --previous` — lit les logs du conteneur qui a planté avant son redémarrage.

- `kubectl -n scoring get inferenceservice scoring-model` — vérifie l'état READY, l'URL et la révision du service.
- `make check-m5` — confronte l'état réel à l'état attendu, item par item.

Poster (QR codes)

- `uvx --from "qrcode[pil]" qr "https://votre-projet-deploye/docs" > docs/qr-projet.png` — génère le QR code du projet déployé.
- `uvx --from "qrcode[pil]" qr "https://github.com/votre-user/mlops-cours-2026" > docs/qr-repo.png` — génère le QR code du repo.

Rattraper un lab (FAQ)

- `git fetch upstream` — récupère les nouveautés du repo du cours (corrections).
- `git switch -c lab/m4-rattrapage` — crée la branche de reprise.
- `git checkout upstream/solution/m4 -- packaging/` — ne récupère que les fichiers du lab concerné.
- `git commit -m "lab/m4 : reprise de la correction m4"` — trace la reprise dans l'historique.

Vérification automatique

- `make check-m10` — pré-contrôle du jalon « final » (tout).
- `make check-m10 JALON=j1` — ce qui est attendu au jalon J1.
- `./labs/m10-capstone/check.sh --sans-cluster` — contrôle repo seul (docs, Git, tags).
- `./labs/m10-capstone/check.sh --gh` — contrôle étendu à la relecture des PR `ext/*` via `gh`.

Pièges / points d'attention

- Ne jouer au jeu du chaos que sur des éléments versionnés ou recréables (git, Argo CD, make) ; jamais `make down`, ni suppression de PVC ou de bucket.
- `dvc pull` ne suffit pas : le remote DVC est propre à chaque poste ; il faut `dvc repro` (puis `dvc push`) pour reconstruire.
- Un tag poussé ne se déplace pas : `git tag -f` et `git push -f` sont refusés ; corriger par un commit sur `main`.
- Ne pas activer « Require a pull request before merging » : le bot GitOps du Lab 6 commite directement sur `main`.
- Décocher « Allow rebase merging » : un rebase effacerait le nom de branche `ext/<nom>` de l'historique.
- Rien en dur : tout (colonnes, cible, groupes sensibles, seuils) doit venir de `dataset.yaml` ou d'un fichier de configuration versionné.
- Aucun secret dans Git (`.env`, `.dvc/config.local`, jetons) ; un secret poussé doit être changé, pas seulement supprimé.
- Aucune donnée personnelle : données publiques uniquement (credit scoring ou dataset validé).
- Le service cloud peut descendre à zéro instance : appeler `/health` quelques minutes avant la soutenance ou fixer une instance minimale.
- Créer une alerte de budget dès l'ouverture du compte cloud et supprimer les ressources après évaluation.
- Répéter la démo au moins trois fois, dont une sur un poste vierge (`git clone` + `make up` + `uv sync` + `dvc repro` + `dvc push`).

- Ne pas fusionner l'extension la veille : sans relecture ni tests, elle casse souvent le socle et les checks.
- Le pré-contrôle vérifie que les livrables existent, il ne note pas la qualité ; un pré-contrôle rouge au tag final coûte des points.

Fiche générée à partir de Lab10-Enonce.pdf et Lab10-Commandes.pdf — cours MLOps.